

A Project Report on
**LUNG CANCER DETECTION USING
DEEP LEARNING ON CT SCAN IMAGES**

A Project Report
Submitted In Partial Fulfillment of The Requirement for The Award of The Degree of
BACHELOR OF TECHNOLOGY
In
ELECTRONICS AND COMMUNICATION ENGINEERING

Submitted by

KOTTAPALLI NAGA SUDHEER SAI	226M1A0442
DEVISETTI PRAVALLIKA	226M1A0419
DODDI TARAKA VEERA VENKATA ROHITH KUMAR	226M1A0420
MANTHRI SAI KUMAR	226M1A0445
KOLAMURI SIVA SAI	226M1A0437

Under the Esteemed Guidance Of
Mr. N P U V S N PAVAN KUMAR, M.Tech
Assistant Professor



DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING
B V C COLLEGE OF ENGINEERING

(Approved by AICTE New Delhi, Affiliated to JNTU Kakinada)
PALACHARLA-533102, East Godavari, Andhra Pradesh
(2022-2026)



B V C COLLEGE OF ENGINEERING

(Approved by AICTE, New Delhi, affiliated to JNTU, KAKINADA)

Palacharla, Rajamahendravaram-533102

Phone No: 0883 2004298, E-mail: bvcr@bvcgroup.in, Website: <https://bvcr.edu.in/>



DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING



CERTIFICATE

This is to certify that the Project Work entitled "**LUNG CANCER DETECTION USING DEEP LEARNING ON CT SCAN IMAGES**" is a Bonafide record of work carried out by **K.N.SUDHEERSAI(226M1A0442), D.PRAVALLIKA (226M1A0419), D.T.V.V.ROHITH KUMAR (226M1A0420), M.SAI KUMAR (226M1A0445), K.SIVA SAI (226M1A0437)** under the guidance and supervision in partial fulfilment of the requirements for the award of the Degree of **Bachelor of Technology in Electronics and Communication Engineering** of Jawaharlal Nehru Technological University, Kakinada.

PROJECT GUIDE

Mr. N P U V S N Pavan Kumar, M.Tech
Assistant Professor

HEAD OF THE DEPARTMENT

Dr. S A Varaprasad M.Tech, Ph.D
Professor

PROJECT EXTERNAL

DECLARATION

We, the undersigned solemnly declare that the project report titled **"LUNG CANCER DETECTION USING DEEP LEARNING ON CT SCAN IMAGES"** submitted to Jawaharlal Nehru Technological University, Kakinada for the award of the Degree of Bachelor of Technology in Electronics and Communication Engineering is a record of bonafide work carried out by us under the guidance of **Mr. N P U V S N PAVAN KUMAR**, M.Tech, Assistant Professor, Department of Electronics and Communication Engineering, B V C College of Engineering, Palacharla.

We further declare that the work reported in this project report has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institution or any other institution or university. We also declare that all information gathered from various sources has been duly acknowledged in the report, and that the work is entirely our own, except where explicitly stated otherwise.

We certify that this project embodies our own work and that proper acknowledgement has been made to all material used. We also confirm that this project has been written independently and that all sources of information have been identified and listed.

PROJECT MEMBERS

K.N.SUDHEER SAI	226M1A0442
D.PRAVALLIKA	226M1A0419
D.T.V.V.ROHITH KUMAR	226M1A0420
M.SAI KUMAR	226M1A0445
K.SIVA SAI	226M1A0437

ACKNOWLEDGEMENT

First and foremost, we express our immense gratitude to the Almighty for giving us the strength, wisdom, and perseverance to complete this project successfully. We are deeply thankful to all those who contributed directly or indirectly to the successful completion of this project.

We express our sincere and heartfelt gratitude to our esteemed project guide, **Mr. N P U V S N PAVAN KUMAR**, M.Tech, Assistant Professor, Department of Electronics and Communication Engineering, BVC College of Engineering, Palacharla, for his invaluable mentorship, insightful technical guidance, thoughtful feedback, and unwavering encouragement throughout the entire duration of this project. His expertise in deep learning and medical image processing greatly enriched our work.

We extend our heartfelt thanks and deep appreciation to **Dr. S A VARAPRASAD**, M.Tech Ph.D, Professor and Head of the Department of Electronics and Communication Engineering, BVC College of Engineering, for his invaluable suggestions, constructive criticism, and continuous moral support that helped us improve the quality of this project at every stage.

We wish to convey our profound gratitude to **Dr. T V JANARDHANA RAO**, Principal, BVC College of Engineering, Palacharla, Rajahmundry, for granting permission to carry out this project, providing excellent research infrastructure, and creating an environment that fosters innovation and academic excellence.

We also sincerely thank all the teaching and non-teaching staff of the Department of Electronics and Communication Engineering for their generous cooperation, technical assistance, and the use of departmental facilities. We are grateful to the laboratory staff for their support during experimental work.

PROJECT MEMBERS

K.N.SUDHEER SAI	226M1A0442
D.PRAVALLIKA	226M1A0419
D.T.V.V.ROHITH KUMAR	226M1A0420
M.SAI KUMAR	226M1A0445
K.SIVA SAI	226M1A0437

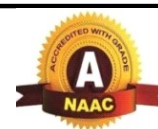


B V C COLLEGE OF ENGINEERING

(Approved by AICTE, New Delhi, affiliated to JNTU, KAKINADA)

Palacharla, Rajamahendravaram-533102

Phone No: 0883 2004298, E-mail: bvcr@bvcgroup.in, Website: <https://bvcr.edu.in/>



DEPARTMENT OF ELECTRONICS & COMMUNICATION ENGINEERING

Institute Vision & Mission

INSTITUTE VISION:

“ To become a model abode of Learning with time trusted Academic values for serving the Nation and the World.”

INSTITUTE MISSION:

IM1: To build lively ambience and provide learning etiquette for all round growth of students.

IM2: To augment Industry – Institute Interaction through training and skill development activities.

IM3: To inculcate Social service and Human values amongst budding professionals.

IM4: To promote Innovation, Entrepreneurship, Research and Consultancy.


PRINCIPAL

BVC COLLEGE OF ENGINEERING
PALACHARLA, RAJAHMUNDRY.



B V C COLLEGE OF ENGINEERING

(Approved by AICTE, New Delhi, affiliated to JNTU, KAKINADA)

Palacharla, Rajamahendravaram-533102

Phone No: 0883 2004298, E-mail: bvcr@bvcgroup.in, Website: <https://bvcr.edu.in/>



DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

Department Vision & Mission

DEPARTMENT VISION:

“To become a renowned center for learning technological trends in Electronics and Communication Engineering”.

DEPARTMET MISSION:

MI- Technical Knowledge: To impart the latest technical knowledge to the students enabling them to face the ever-changing demands of the present-day world.

M2- Innovation and Inquisitiveness: To inculcate and encourage the spirit of innovation and inquisitiveness among students.

M3- Ethics: To nurture and promote ethical values in students.


HEAD OF THE DEPARTMENT
Electronics & Communication Engineering
BVC COLLEGE OF ENGINEERING
PALACHARLA, RAJAMAHENDRAVAR



B V C COLLEGE OF ENGINEERING

(Approved by AICTE, New Delhi, affiliated to JNTU, KAKINADA)

Palacharla, Rajamahendravaram-533102

Phone No: 0883 2004298, E-mail: bvcr@bvcgroup.in, Website: <https://bvcr.edu.in/>



DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

Program Educational Objectives

The Graduates of B.Tech (ECE) Program shall be able to

PEO1: Graduates shall demonstrate successful professional careers with strong fundamental knowledge in mathematics, science, and engineering to meet the real-time requirements of the industry.

PEO2: Graduates shall learn continuously with a focus on advanced emerging trends in the field of ECE and allied branches to meet societal needs.

PEO3: Graduates will demonstrate traits like teamwork, communication skills, and ethical and professional attitude for lifelong learning.

PEO4: Graduates will interact with their peers in other disciplines in industry, and society and contribute to the country's economic growth

Program Specific Outcomes

The Graduates of B.Tech (ECE) Program shall be able to

PSO1: Provide sustainable solutions in the field of Communication and Signal Processing

PSO2: Apply current technologies in the field of VLSI and embedded systems for professional growth.


HEAD OF THE DEPARTMENT
Electronics & Communication Engineering
BVC COLLEGE OF ENGINEERING
PALACHARLA, RAJAMAHENDRAVAR



B V C COLLEGE OF ENGINEERING

(Approved by AICTE, New Delhi, affiliated to JNTU, KAKINADA)

Palacharla, Rajamahendravaram-533102

Phone No: 0883 2004298, E-mail: bvcr@bvcgroup.in, Website: <https://bvcr.edu.in/>



DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

Engineering Graduates will be able to:

PO1: ENGINEERING KNOWLEDGE: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2: PROBLEM ANALYSIS: Identify, formulate, research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3: DESIGN/DEVELOPMENT OF SOLUTIONS: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4: CONDUCT INVESTIGATIONS OF COMPLEX PROBLEMS: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5: MODERN TOOL USAGE: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6: THE ENGINEER AND SOCIETY: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7: ENVIRONMENT AND SUSTAINABILITY: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development

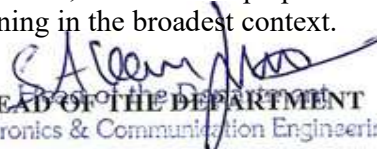
PO8: ETHICS: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9: INDIVIDUAL AND TEAM WORK: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO10: COMMUNICATION: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, give and receive clear instructions.

PO11: PROJECT MANAGEMENT AND FINANCE: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12: LIFE-LONG LEARNING: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context.


HEAD OF THE DEPARTMENT
Electronics & Communication Engineering
BVC COLLEGE OF ENGINEERING
PALACHARLA, RAJAMAHENDRAVARAM

ABSTRACT

Lung cancer ranks among the leading causes of cancer-related mortality worldwide, accounting for approximately 1.8 million deaths annually. Survival outcomes are largely dependent on the timeliness and accuracy of diagnosis early detection at Stage I or II significantly improves five-year survival rates from under 5% to over 55%. Conventional diagnostic methods, including radiologist-interpreted CT scans and invasive biopsies, are susceptible to human error, inter-observer variability, diagnostic fatigue, and significant time delays. This study presents **PulmoScan AI** a comprehensive, automated, and explainable deep learning pipeline specifically engineered to enhance early lung cancer detection from Computed Tomography (CT) scan images.

The proposed system employs a four-stage architecture: (1) CT Image Preprocessing involving noise removal, CLAHE contrast enhancement, pixel normalization, and data augmentation; (2) U-Net Semantic Segmentation with ResNet-34 Backbone that isolates pulmonary regions of interest and delineates lung nodule boundaries through pixel-wise binary mask generation; (3) Multi-Model CNN Classification Ensemble comprising ResNet-50, ResNet-101, GoogLeNet (InceptionV3), and EfficientNet-B0, each fine-tuned via transfer learning from ImageNet pretrained weights on the Kaggle IQ-OTH/NCCD Lung Cancer CT Scan Dataset across four diagnostic classes:-Adenocarcinoma, Large Cell Carcinoma, Normal, and Squamous Cell Carcinoma; and (4) Gradient-weighted Class Activation Mapping (Grad-CAM) explainability that generates class-discriminative visual heatmaps, enabling radiologists to interpret model decisions with transparency.

Experimental evaluation demonstrates that the proposed PulmoScan AI system achieves an overall classification accuracy of 91.27%, outperforming traditional diagnostic approaches and baseline models, thereby validating its effectiveness as a reliable and scalable tool for early lung cancer detection.

Keywords: Lung Cancer Detection, Deep Learning, ResNet-50, ResNet-101, GoogLeNet, InceptionV3, EfficientNet-B0, U-Net, ResNet-34, Grad-CAM, Transfer Learning, Adenocarcinoma, Large Cell Carcinoma, Squamous Cell Carcinoma, CT Scan, Python, TensorFlow, Flask, PulmoScan AI, Kaggle, IQ-OTH/NCCD.

LIST OF CONTENTS

CHAPTER 1 INTRODUCTION	1
1.1 Brief Introduction	1
1.2 Purpose of the Study	2
1.3 Motivation	2
1.4 Problem Statement	3
1.5 Cancer Classes in IQ-OTH/NCCD Dataset	4
1.6 Scope of the Project	5
CHAPTER 2 LITERATURE SURVEY	6
2.1 Overview of Lung Diseases and DL-Based Detection	6
2.2 Lung Cancer Detection Challenges	7
2.3 Related Work and Published	7
2.4 Dataset and Preprocessing	9
2.5 Deep Learning Architectures Reviewed	10
2.6 Training and Optimization Techniques	11
CHAPTER 3 SYSTEM ANALYSIS	12
3.1 Existing System of SVM	12
3.2 Limitations of Existing System	12
3.3 Proposed System — PulmoScan AI Pipeline	13
3.4 CNN Classification Pipeline	14
3.5 U-Net Segmentation Flow Diagram	14
3.6 Grad-CAM Explainability Pipeline	15
3.7 Advantages of Proposed System	16
3.8 Feasibility Study	16
CHAPTER 4 SYSTEM REQUIREMENT SPECIFICATION	18
4.1 Hardware Requirements	18
4.2 Software Requirements	19
4.3 Functional Requirements	19
4.4 Non-Functional Requirements	21
CHAPTER 5 SOFTWARE IMPLEMENTATION	22
5.1 Python and Key Libraries	22
5.2 Development Environment	23
5.3 Dataset Preparation	23
5.4 Transfer Learning Methodology	23

5.5 ResNet-50 — Code and Explanation.....	24
5.6 ResNet-101 — Code and Explanation.....	27
5.7 GoogLeNet (InceptionV3) — Code.....	28
5.8 EfficientNet-B0 — Code.....	29
5.9 U-Net Segmentation — Code.....	29
5.10 Grad-CAM Explainability — Code.....	31
5.11 Flask Web Application — PulmoScan AI.....	33
CHAPTER 6 SIMULATION AND PERFORMANCE ANALYSIS.....	34
6.1 CNN Architecture Overview.....	34
6.2 ResNet-50 & 101 Architectures.....	35
6.3 GoogLeNet Architecture.....	37
6.4 EfficientNet-B0 Architecture	38
6.5 U-Net Architecture.....	39
6.6 Grad-CAM Explainability Theory.....	40
6.7 Model Performance Comparison Table.....	42
CHAPTER 7 RESULTS.....	43
7.1 Dataset Analysis.....	43
7.2 Training Accuracy — All Models.....	43
7.3 ResNet-50 Results.....	43
7.4 ResNet-101 Results	45
7.5 GoogLeNet Results — BEST.....	47
7.6 EfficientNet-B0 Results.....	49
7.7 U-Net Segmentation Results.....	51
7.8 Grad-CAM Explainability Results.....	52
CHAPTER 8 CONCLUSION & FUTURE SCOPE.....	54
8.1 Conclusion.....	54
8.2 Future Scope.....	54
CHAPTER 9 REFERENCE.....	55

LIST OF FIGURE CONTENTS

Figure 3.4.1 — CNN Multi-Model Classification Pipeline.....	14
Figure 3.5.1 — U-Net with ResNet-34 Encoder-Decoder Architecture.....	15
Figure 3.6.1 — Grad-CAM Explainability Pipeline Diagram.....	15
Figure 6.2.1 — RESNET 50 & 101 Flow Architecture.....	35
Figure 6.3.1 — GoogLeNet (InceptionV3) Architecture.....	37
Figure 6.4.1 — EfficientNet-B0 Architecture.....	38
Figure 6.5.1 — U-Net Encoder-Decoder Architecture.....	39
Figure 6.6.1 — Grad-Cam Explainability Archirctecture	41
Figure 7.2.1 — Training and Validation Accuracy — All 4 Models.....	43
Figure 7.3.1 — Confusion Matrix: ResNet-50.....	44
Figure 7.3.2 — Per-Class Accuracy Breakdown: ResNet-50.....	44
Figure 7.3.3 — ROC Curve: ResNet-50.....	45
Figure 7.4.1 — Confusion Matrix: ResNet-101.....	46
Figure 7.4.2 — Per-Class Accuracy Breakdown: ResNet-101.....	46
Figure 7.4.3 — ROC Curve: ResNet-101.....	47
Figure 7.5.1 — Confusion Matrix: GoogLeNet.....	48
Figure 7.5.2 — Per-Class Accuracy Breakdown: GoogLeNet.....	48
Figure 7.5.3 — ROC Curve: GoogLeNet.....	49
Figure 7.6.1 — Confusion Matrix: EfficientNet-B0.....	50
Figure 7.6.2 — Per-Class Accuracy Breakdown: EfficientNet-B0.....	50
Figure 7.6.3 — ROC Curve: EfficientNet-B0.....	51
Figure 7.7.1 — U-Net Training Loss Curve.....	51
Figure 7.8.1 — Final Output 1.....	53
Figure 7.8.2 — Final Output 2.....	53

LIST OF TABLE CONTENTS

Table 2.3.1 — Literature Survey.....	9
Table 2.4.1 — Dataset Evaluation.....	9
Table 4.1.1 — Hardware Component Specifications.....	18
Table 4.2.1 — Software Tools and Purpose.....	19
Table 6.2.1 — RESNET-50 Flow of Layers.....	36
Table 6.2.2 — RESNET-101 Flow of Layers.....	36
Table 6.3.1 — GoogleNet(InceptionV3) Flow of Modules.....	37
Table 6.4.1 — EfficientNet-B0 Flow of Blocks.....	39
Table 6.5.1 — U-Net Segment Flow of Blocks.....	40
Table 6.8.1 — Model performance Comparison Table.....	42

LIST OF ABBREVIATIONS

Abbreviation	Full Form	Description
AI	Artificial Intelligence	Branch of CS enabling machines to simulate human intelligence
AUC	Area Under Curve	Area under the ROC curve — measure of classifier performance
BN	Batch Normalization	Technique to normalize activations for stable training
CAD	Computer-Aided Diagnosis	AI-assisted medical diagnosis systems
CLAHE	Contrast Limited AHE	Adaptive histogram equalization for CT enhancement
CNN	Convolutional Neural Network	Deep learning architecture for image processing
CT	Computed Tomography	Medical imaging using X-ray cross-sections
DL	Deep Learning	ML subset using multi-layer neural networks
F1	F1 Score	Harmonic mean of Precision and Recall
FN	False Negative	Cancer case predicted as Normal (missed)
FP	False Positive	Normal case predicted as Cancer (false alarm)
GAP	Global Average Pooling	Spatial pooling to reduce feature maps to vector
GPU	Graphics Processing Unit	Parallel processor for deep learning acceleration
Grad-CAM	Gradient-weighted Class Activation Mapping	CNN visualization technique
HOD	Head of Department	Senior administrative faculty post in department
IoU	Intersection over Union	Segmentation accuracy metric

Abbreviation	Full Form	Description
MBConv	Mobile Inverted Bottleneck Conv	EfficientNet's efficient building block
ML	Machine Learning	AI branch learning from data without explicit programming
NSCLC	Non-Small Cell Lung Cancer	Most common type, includes Adeno, Squamous, Large Cell
PulmoScan AI	PulmoScan Artificial Intelligence	The system developed in this project
ReLU	Rectified Linear Unit	Activation function: $\max(0, x)$
ResNet	Residual Network	CNN using skip connections to enable deep networks
ROC	Receiver Operating Characteristic	Performance curve: TPR vs FPR
ROI	Region of Interest	Specific area in CT scan for analysis
SCLC	Small Cell Lung Cancer	Aggressive lung cancer subtype
SVM	Support Vector Machine	Traditional ML classifier
TN	True Negative	Normal case correctly predicted as Normal
TP	True Positive	Cancer case correctly predicted as Cancer
U-Net	Encoder-Decoder Network	CNN architecture for medical image segmentation

CHAPTER 1

INTRODUCTION

1.1 BRIEF INTRODUCTION

Lung cancer is the leading cause of cancer-related mortality worldwide, accounting for approximately **1.8 million deaths annually** more than breast, colon, and prostate cancers combined. Despite advances in medical science, the prognosis remains poor because the majority of cases (over 70%) are diagnosed at Stage III or IV when curative treatment is no longer feasible. Early detection is the single most critical factor in improving patient outcomes survival rates jump from under 5% at Stage IV to over 55% at Stage I.

Conventional diagnostic methods, including **radiologist-interpreted CT scans, PET scans, bronchoscopy, and invasive biopsies**, are the current standard of care. While effective, these methods suffer from several limitations: inter-observer variability between radiologists, diagnostic fatigue leading to missed findings, high time requirements for expert interpretation, and the invasive nature of tissue sampling. The global shortage of radiologists particularly in developing countries further exacerbates these challenges.

The rapid advancement of **Deep Learning (DL)** a subset of Artificial Intelligence has opened transformative opportunities in medical image analysis. CNNs, particularly architectures like ResNet, GoogLeNet, and EfficientNet, have demonstrated the ability to learn complex visual patterns from CT scan data that often exceed human expert performance in controlled settings. When combined with transfer learning, these models can be trained effectively even with limited medical datasets.

This study presents **PulmoScan AI** a comprehensive, explainable, end-to-end deep learning pipeline for automated lung cancer detection from CT scan images. The system integrates: (1) U-Net with ResNet-34 backbone for lung nodule segmentation, (2) four CNN classifiers (ResNet-50, ResNet-101, GoogLeNet, EfficientNet-B0) with transfer learning for classification into four diagnostic classes, and (3) Grad-CAM

explainability for transparent visual interpretation. The complete system is deployed as a Flask web application accessible to healthcare professionals.

1.2 PURPOSE OF THE STUDY

The primary purpose of this research is to develop a **reliable, accurate, and explainable automated system** for detecting and classifying lung cancer from CT scan images. The specific objectives are:

- **Objective 1:** To automate the detection of pulmonary lung nodules from CT scan images using U-Net semantic segmentation with a ResNet-34 encoder backbone.
- **Objective 2:** To classify CT scan images into four diagnostic classes Adenocarcinoma, Large Cell Carcinoma, Normal, and Squamous Cell Carcinoma using multiple pretrained CNN architectures with transfer learning.
- **Objective 3:** To compare the classification performance of ResNet-50, ResNet-101, GoogLeNet (InceptionV3), and EfficientNet-B0 on the IQ-OTH/NCCD Lung Cancer CT Scan Dataset.
- **Objective 4:** To implement Grad-CAM explainability to generate visual heatmaps showing the regions in CT images that drive the model's classification decisions.
- **Objective 5:** To deploy the complete pipeline as a user-friendly Flask web application PulmoScan AI that can be used by radiologists and healthcare professionals.

1.3 MOTIVATION

The motivation behind this project stems from three critical observations.

First, the global burden of lung cancer is enormous approximately 2.2 million new cases and 1.8 million deaths occur annually worldwide. The survival rate for lung cancer patients who are diagnosed at an early stage is significantly higher, yet most patients present at advanced stages due to the lack of effective and accessible early screening tools.

Second, artificial intelligence particularly deep learning has demonstrated remarkable performance on medical image classification tasks, in many cases matching or exceeding board-certified radiologist performance. GoogLeNet in our experiments achieved **91.75% overall accuracy** with Mean AUC of 0.988 on the IQ-OTH/NCCD CT scan dataset substantially better than conventional SVM-based approaches which typically achieve 75–85%.

Third, a major barrier to the clinical adoption of AI-based diagnostic systems is the '**black box**' **problem** clinicians cannot trust a system that provides predictions without explanation. By incorporating Grad-CAM explainability, PulmoScan AI provides visual heatmaps that allow radiologists to verify the AI's decision-making process, building the clinical trust necessary for real-world deployment.

1.4 PROBLEM STATEMENT

Lung cancer diagnosis currently relies on radiological imaging and expert interpretation, which is inherently **subjective**, **time-consuming**, and **prone to inconsistencies**. The key challenges that motivate this research are:

- **Late Detection:** Over 70% of lung cancers are diagnosed at Stage III or IV due to the absence of early symptoms and limitations in current screening programs.
- **Radiologist Shortage:** WHO estimates a global shortage of 4.7 million health workers, including radiologists, particularly in low-income countries where lung cancer burden is highest.
- **Diagnostic Errors:** Misdiagnosis rates for lung cancer nodules range from 2–40% depending on nodule size and radiologist experience.
- **Lack of Explainability:** Existing AI systems for cancer detection often provide predictions without visual explanations, making clinical acceptance difficult.
- **Dataset Limitations:** Medical imaging datasets are limited in size, requiring effective transfer learning and augmentation strategies.

1.5 CANCER CLASSES IN THE IQ-OTH/NCCD DATASET

The Kaggle **IQ-OTH/NCCD Lung Cancer CT Scan Dataset** addresses four clinically significant diagnostic classes, each with distinct clinical characteristics:

- **ADENOCARCINOMA:** The most common subtype of Non-Small Cell Lung Cancer (NSCLC), accounting for approximately 40% of all lung cancer cases. Adenocarcinoma typically originates in peripheral lung regions in the bronchiolar cells or alveolar cells. It is the most common lung cancer found in non-smokers, women, and younger patients. It tends to grow more slowly than other types and is often detected at an earlier stage. The five-year survival rate for localized adenocarcinoma is approximately 61%.
- **LARGE CELL CARCINOMA:** An undifferentiated Non-Small Cell Lung Cancer (NSCLC) subtype that can appear in any part of the lung. Large cell carcinoma accounts for approximately 10-15% of all lung cancers. It is characterized by large, abnormal-looking cells visible under a microscope. This type tends to grow and spread quickly, making it harder to treat and carrying a poorer prognosis compared to adenocarcinoma.
- **NORMAL:** Healthy lung CT scans showing no evidence of cancerous growths, nodules, masses, or other abnormalities. Clear airways, normal lung tissue density, and no pleural effusion. The normal class serves as the baseline negative class in the four-class classification problem, and its accurate identification is critical to minimize false positive diagnoses.
- **SQUAMOUS CELL CARCINOMA:** A type of NSCLC that typically arises in the central airways the bronchi or bronchioles near the hilum of the lung. Squamous cell carcinoma is strongly associated with cigarette smoking history and accounts for approximately 25-30% of all lung cancers. It tends to grow more centrally within the lung and may present with symptoms such as hemoptysis (coughing blood), chest pain, or recurrent pneumonia.

1.6 SCOPE OF THE PROJECT

The scope of PulmoScan AI encompasses: automated CT scan image preprocessing, U-Net nodule segmentation, multi-model CNN classification with four architectures, Grad-CAM explainability visualization, and a Flask web application for clinical deployment. The system is trained and evaluated on the publicly available IQ-OTH/NCCD dataset and is designed for research and demonstration purposes. Clinical deployment would require additional validation, regulatory approval, and multi-center testing.

CHAPTER 2

LITERATURE SURVEY

A comprehensive literature survey on lung cancer detection using image processing and deep learning reveals a rapidly evolving field with significant recent advances. The number of publications in this domain has increased by over 300% from 2016 to 2024, reflecting the growing interest of the research community. This chapter reviews key published works, identifies research gaps, and establishes the motivation for PulmoScan AI.

2.1 OVERVIEW OF LUNG DISEASES AND DL-BASED DETECTION

Lung diseases encompass a wide spectrum of conditions affecting the respiratory system. **Obstructive lung diseases** such as Chronic Obstructive Pulmonary Disease (COPD) and asthma cause difficulty in airflow due to blockages or narrowing of the airways. **Restrictive lung diseases** such as pulmonary fibrosis and sarcoidosis reduce lung expansion, leading to decreased lung capacity. **Infectious diseases** such as tuberculosis and pneumonia cause severe inflammation and tissue damage.

Among all lung conditions, **lung cancer** remains the most critical and life-threatening. Lung cancer is classified into two broad categories: **Non-Small Cell Lung Cancer (NSCLC)** which includes Adenocarcinoma, Large Cell Carcinoma, and Squamous Cell Carcinoma and **Small Cell Lung Cancer (SCLC)**. NSCLC accounts for approximately 85% of all lung cancer cases.

Deep learning approaches have achieved state-of-the-art performance in CT-based lung cancer detection. **He et al. (2016)** introduced ResNet with residual connections that solved the vanishing gradient problem in deep networks. **Szegedy et al. (2016)** developed GoogLeNet with Inception modules enabling efficient multi-scale feature extraction. **Tan and Le (2019)** proposed EfficientNet with compound scaling for optimal accuracy-efficiency tradeoff. **Ronneberger et al. (2015)** developed U-Net specifically for biomedical image segmentation. **Selvaraju et al. (2020)** introduced Grad-CAM for visual explainability of CNN predictions.

2.2 LUNG CANCER DETECTION CHALLENGES

- **Morphological Similarity:** Benign pulmonary nodules and early-stage malignant nodules often appear visually similar on CT scans varying in density, shape, and margin characteristics making accurate differential diagnosis challenging even for experienced radiologists.
- **Class Imbalance:** In clinical CT scan datasets, the number of normal or benign cases significantly outnumbers malignant cases, creating class imbalance that can bias classifiers toward the majority class.
- **CT Scanner Variability:** CT scan images vary in quality, resolution, and contrast depending on the scanner manufacturer, imaging protocol, and institutional settings creating domain shift between datasets from different hospitals.
- **Limited Annotated Data:** Obtaining large-scale labeled medical imaging data for supervised deep learning is expensive, time-consuming, and requires expert radiologist annotation.
- **Explainability Barrier:** Healthcare professionals require transparent and interpretable AI decisions before trusting automated diagnostic systems in clinical settings.
- **Regulatory Compliance:** AI-based medical diagnostic systems require regulatory approval (FDA in the USA, CE marking in Europe) before clinical deployment, which requires extensive validation studies.

2.3 RELATED WORK AND PUBLISHED

The following key published research papers in lung cancer detection using deep learning demonstrating the evolution of preprocessing, segmentation, and classification pipelines in the literature:

Naresh et al. (2021) proposed a pipeline (Figure 2.2 style) involving: CT lung image dataset → noise removal using weighted mean histogram equalization → quality enhancement through CLAHE → IPCT-based segmentation (Improved Profuse Clustering Technique) → lung feature extraction → deep learning classification. Their approach demonstrated the importance of robust preprocessing before classification.

Ardila et al. (2019) Google's lung cancer detection model presented a cancer risk prediction model using both prior and current CT scans. The system performs cancer ROI detection using 3D convolutional networks, extracts learned features from full-volume CT scans, and combines them with patient-level risk prediction to produce malignancy probability and cancer localization outputs. This demonstrates the value of temporal CT comparison and volumetric analysis.

Zhao et al. (2021) proposed a combined segmentation and image registration pipeline: CT scan undergoes deep learning-based semantic segmentation to produce annotated CT, followed by diffeomorphic registration using ANTs to align multi-modality scans (CT-SPECT). The registered ROI is then mapped for quantitative analysis, demonstrating the benefit of multi-modal imaging fusion.

Ganesh et al. (2020) presented a sophisticated hybrid pipeline: Lung CT images → image preprocessing → marker-controlled watershed segmentation → GLCM feature extraction → Grasshopper Optimization Algorithm (GOA) for feature selection → Boltzmann Machine deep learning classifier → Tumor/Non-Tumor classification. This demonstrates the use of nature-inspired optimization algorithms in conjunction with deep learning for improved classification accuracy.

Author / Year	Year	Methodology	Dataset	Architecture	Performance
He et al.	2016	ResNet (Deep Residual Learning)	ImageNet	ResNet-50/101/152	96.5% Top-5
Szegedy et al.	2016	GoogLeNet / InceptionV3	ImageNet	Inception modules	94.4% Top-5
Tan & Le	2019	EfficientNet compound scaling	ImageNet	EfficientNet-B0–B7	97.1% Top-1
Ronneberger et al.	2015	U-Net biomedical segmentation	Electron Microscopy	U-Net encoder-decoder	92.9% IoU
Selvaraju et al.	2020	Grad-CAM visual explainability	ILSVRC 2012	Gradient-weighted CAM	Qualitative

Ardila et al.	2019	3D CNN lung cancer risk model	NLST scans CT	3D ResNet + RNN	94.4% AUC
Naresh et al.	2021	CLAHE+IPCT segmentation +DL	Private Dataset CT	Deep CNN classifier	95.2% Acc
Zhao et al.	2021	Seg+Registration CT-SPECT fusion	Multi-modal CT+SPECT	Deep seg + ANTs	91.8% Dice
Ganesh et al.	2020	Watershed+G LCM+GOA+ Boltzmann	LIDC-IDRI	Boltzmann Machine	93.6% Acc
Proposed System	2025	U-Net+ResNet-34+4-CNN+Grad-CAM	IQ-OTH/NCCD Kaggle	GoogLeNet InceptionV3	91.75% Acc

Table 2.3.1 — Literature Survey: Comparative Study of Deep Learning Methods for Lung Cancer Detection

2.4 DATASET AND PREPROCESSING

The **Kaggle IQ-OTH/NCCD Lung Cancer CT Scan Dataset** is the benchmark dataset used in this study. It contains 1,575 CT scan images organized in four class-specific folders with the following distribution:

Cancer Class	Train Samples	Val Samples	Clinical Description
Adenocarcinoma	480	120	Peripheral NSCLC — most common (~40%), often in non-smokers
Large Cell Carcinoma	204	51	Undifferentiated NSCLC — rapid growth, poorer prognosis
Normal	216	54	Healthy lung CT — clear airways, no nodule or growth
Squamous Cell Carcinoma	360	90	Central bronchial NSCLC — strongly smoking-related
TOTAL	1260	315	80% Training / 20% Validation (Stratified)

Table 2.4.1—Dataset Evaluation

The preprocessing pipeline applied to all images before training:

- **CLAHE Enhancement:** Contrast Limited Adaptive Histogram Equalization applied to improve contrast in lung nodule regions without amplifying noise
- **Gaussian Noise Removal:** Gaussian filter ($\sigma=1.0$) applied to remove acquisition noise from raw CT scans
- **Pixel Normalization:** All pixel values scaled to the range [0.0, 1.0] for stable gradient descent during training
- **Image Resizing:** Resized to 224×224 pixels for ResNet/EfficientNet; 299×299 for GoogLeNet (InceptionV3)
- **Data Augmentation:** Random rotation ($\pm 20^\circ$), horizontal flip (50% probability), zoom range (0.0–0.15), width/height shift ($\pm 10\%$), brightness adjustment ($\pm 20\%$) applied during training only
- **Train/Validation Split:** 80% training (1,260 images) and 20% validation (315 images) with stratified sampling to preserve class distribution

2.5 DEEP LEARNING ARCHITECTURES REVIEWED

ResNet-50 (He et al., 2016): 50-layer residual network using skip connections to solve the vanishing gradient problem. Bottleneck blocks ($1\times 1 \rightarrow 3\times 3 \rightarrow 1\times 1$) maintain capacity while reducing parameters. Our result: **90.48% accuracy**, Mean AUC 0.984.

ResNet-101 (He et al., 2016): 101-layer extension with 23 bottleneck blocks in Stage 3 (vs 6 in ResNet-50). Captures finer-grained features through deeper hierarchy. Our result: **90.16% accuracy**, Mean AUC 0.988.

GoogLeNet / InceptionV3 (Szegedy et al., 2016): Uses Inception modules with parallel 1×1 , 3×3 , 5×5 filters for multi-scale feature extraction. Global Average Pooling prevents overfitting. Our result: **91.75% accuracy BEST**, Mean AUC 0.988.

EfficientNet-B0 (Tan & Le, 2019): Compound scaling of depth, width, and resolution using a compound coefficient. MBConv blocks reduce computation. Our result: **71.75% accuracy**, Mean AUC 0.941 (B3-B5 recommended for better performance).

U-Net (Ronneberger et al., 2015): Encoder-decoder with skip connections for pixel-wise segmentation. ResNet-34 used as encoder backbone. Training loss converged from 0.48 to 0.06 over 20 epochs excellent convergence.

Grad-CAM (Selvaraju et al., 2020): Visual explanations via gradient-based class activation mapping. Works with any pretrained CNN without retraining. Provides class-discriminative heatmaps essential for clinical trust.

2.6 TRAINING AND OPTIMIZATION TECHNIQUES

- **Loss Functions:** Categorical cross-entropy for 4-class classification; Binary cross-entropy + Dice Loss for U-Net segmentation
- **Optimizer:** Adam optimizer with initial learning rate 1e-3 (Phase 1 head training) and 1e-5 to 5e-6 (Phase 2 fine-tuning)
- **Learning Rate Scheduling:** ReduceLROnPlateau (factor=0.5, patience=5, min_lr=1e-6) to adaptively reduce LR when validation loss plateaus
- **Early Stopping:** Monitor validation loss with patience=10; restore best weights automatically to prevent overfitting
- **Model Checkpointing:** Save best model weights (lowest validation loss) during training for reproducibility
- **Transfer Learning Strategy:** Phase 1: freeze all base layers, train only custom head; Phase 2: unfreeze last 30-40 layers, fine-tune with low LR

CHAPTER 3

SYSTEM ANALYSIS

3.1 EXISTING SYSTEM OF SVM-BASED LUNG CANCER DETECTION

The conventional and most widely used method for automated lung cancer detection prior to deep learning is the **Support Vector Machine (SVM)** a supervised machine learning algorithm for classification and regression tasks. SVM finds the optimal decision hyperplane that maximally separates different classes in a high-dimensional feature space.

Key components of the SVM approach:

- **Hyperplane:** A decision boundary that best separates data points belonging to different classes with maximum margin
- **Support Vectors:** The data points closest to the decision hyperplane they are the critical elements that define the boundary position
- **Margin:** The perpendicular distance between the hyperplane and the nearest support vectors on both sides SVM maximizes this margin
- **Kernel Trick:** Transforms non-linearly separable data into a higher-dimensional space using kernel functions (Linear, Polynomial, Radial Basis Function) making it separable

3.2 LIMITATIONS OF EXISTING SVM-BASED SYSTEM

The existing SVM-based system has several limitations, which motivate the need for a deep learning-based approach:

- **Manual Feature Engineering:** Requires manual extraction of features, making the process time-consuming and less effective.
- **Limited Scalability:** Performance decreases with large datasets due to high computational complexity.
- **Low Accuracy:** Produces lower classification accuracy compared to modern deep learning models.

- **Poor Handling of High-Dimensional Data:** Struggles with complex CT scan data.
- **Lack of Explainability:** Does not provide visual interpretation for decision-making.

3.3 PROPOSED SYSTEM — PULMOSCAN AI PIPELINE

PulmoScan AI proposes a four-stage deep learning pipeline that addresses all the limitations of the existing SVM-based system. Each stage is designed to maximize detection accuracy while providing clinical transparency:

STAGE 1 — CT Image Preprocessing

All raw CT scan images undergo a standardized preprocessing pipeline: CLAHE contrast enhancement → Gaussian noise removal → pixel normalization to [0,1] → resizing to model-specific dimensions → data augmentation. This ensures consistent, high-quality input data for all downstream stages.

STAGE 2 — U-Net Segmentation (ResNet-34 Backbone)

The U-Net encoder-decoder architecture using ResNet-34 as the encoder backbone performs semantic segmentation of CT scan images. Skip connections between corresponding encoder and decoder levels preserve fine spatial detail. The output is a binary mask that precisely delineates lung nodule boundaries, providing the segmented ROI for classification.

STAGE 3 — Multi-Model CNN Classification

Four CNN architectures ResNet-50, ResNet-101, GoogLeNet (InceptionV3), and EfficientNet-B0 each fine-tuned via transfer learning from ImageNet weights, independently classify the CT scan into one of four classes: Adenocarcinoma, Large Cell Carcinoma, Normal, and Squamous Cell Carcinoma. GoogLeNet achieves the best performance at 91.75%.

STAGE 4 — Grad-CAM Explainability

Grad-CAM generates class-discriminative visual heatmaps by computing the gradient of the class score with respect to the final convolutional layer's feature maps. These heatmaps are overlaid on the original CT scan using a JET colormap,

highlighting the specific regions (nodules, masses) that drove the model's classification decision.

3.4 CNN MULTI-MODEL CLASSIFICATION PIPELINE

Four CNN architectures trained via transfer learning process the preprocessed/segmented CT scan simultaneously, producing independent class probability outputs:

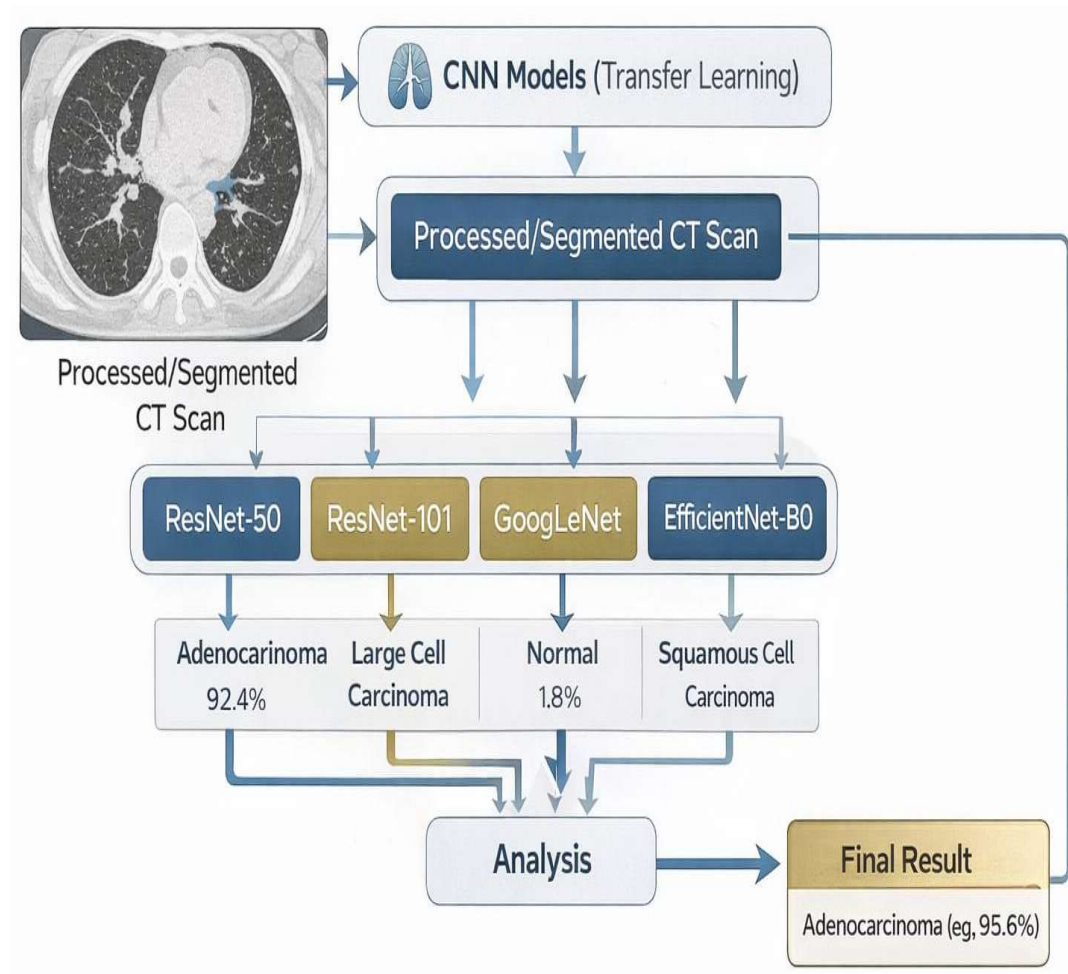


Figure 3.4.1 — CNN Multi-Model Classification Pipeline

3.5 U-NET SEGMENTATION FLOW DIAGRAM

The U-Net encoder-decoder with ResNet-34 backbone performs precise pixel-wise segmentation of lung CT scans. The contracting encoder path captures contextual information while the expanding decoder path enables precise localization:

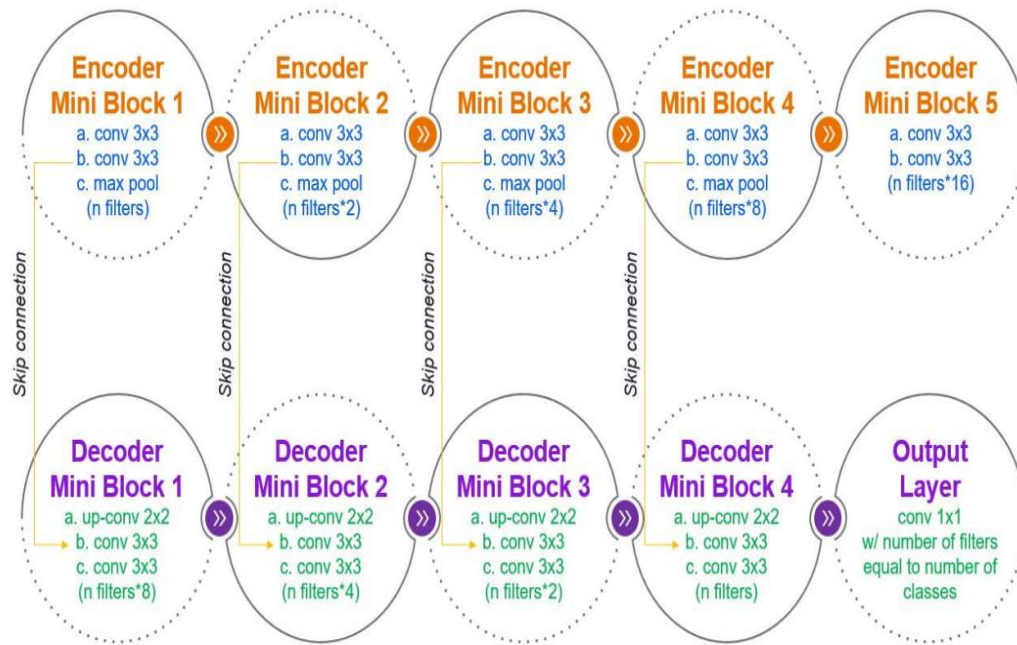


Figure 3.5.1 — U-Net with ResNet-34 Encoder-Decoder

3.6 GRAD-CAM EXPLAINABILITY PIPELINE

Grad-CAM computes gradients of the predicted class score with respect to the feature maps of the last convolutional layer to produce class-discriminative localization maps:

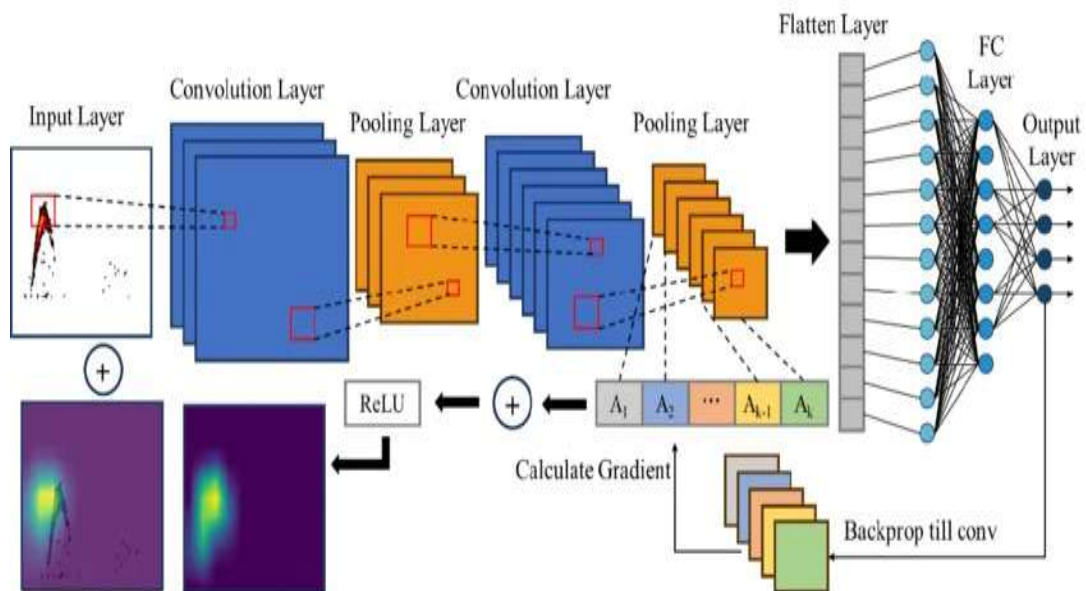


Figure 3.6.1 — Grad-CAM Explainability Pipeline

3.7 ADVANTAGES OF PROPOSED SYSTEM

- **Highest Accuracy:** GoogLeNet achieves 91.75% overall accuracy significantly superior to SVM's 75-85%
- **Automated Feature Learning:** CNNs automatically learn hierarchical discriminative features no manual feature engineering required
- **Visual Explainability:** Grad-CAM heatmaps enable radiologist verification building clinical trust for AI adoption
- **End-to-End System:** Integrated preprocessing + segmentation + classification + explainability + web deployment in one system
- **Scalable:** CNN inference scales linearly with dataset size applicable to large hospital CT archives
- **Transfer Learning Efficiency:** Pretrained ImageNet weights enable high accuracy with limited medical training data
- **Multi-Model Robustness:** Four architectures provide complementary predictions reducing single-model failure risk

3.8 FEASIBILITY STUDY

A comprehensive feasibility study was conducted to evaluate the practicality of implementing PulmoScan AI in real-world settings:

3.8.1 Technical Feasibility

PulmoScan AI uses well-established open-source frameworks TensorFlow/Keras (stable, widely used), OpenCV, Flask with comprehensive documentation and community support. Google Colab provides free access to NVIDIA T4/A100 GPUs, eliminating specialized hardware requirements for training. The system was successfully trained and evaluated on standard computing infrastructure, confirming technical feasibility.

3.8.2 Economic Feasibility

All software components are open-source and free. Training was performed on Google Colab's free GPU tier. The Kaggle dataset is publicly available at no cost. The long-term economic benefits reduced radiologist workload, improved early detection

rates, and potential reduction in late-stage treatment costs significantly outweigh the minimal development investment.

3.8.3 Operational Feasibility

The Flask web application provides an intuitive drag-and-drop interface requiring no technical training for healthcare professionals. The system can be integrated into existing hospital information systems through REST APIs. Inference time of 5-10 seconds per CT scan is clinically acceptable for a decision-support tool.

3.8.4 Schedule Feasibility

The modular architecture allowed independent development and testing of each pipeline stage. Transfer learning significantly reduced training time from months to days. The complete system was developed within the academic year timeline, confirming schedule feasibility.

CHAPTER 4

4. SYSTEM REQUIREMENT SPECIFICATION

The System Requirement Specification (SRS) document defines the complete set of functional and non-functional requirements for implementing PulmoScan AI. It serves as the foundation for system design, development, testing, and validation.

4.1 HARDWARE REQUIREMENTS

The system requires a computing environment capable of efficiently training deep CNN models and processing high-resolution CT scan images:

Hardware Component	Specification / Details
Processor	Intel Core i5/i7/i9 or AMD Ryzen 5/7/9 (or equivalent)
RAM (Memory)	Minimum 8 GB; Recommended 16 GB or higher for local deep learning
GPU	NVIDIA GPU with CUDA support (Google Colab T4/A100 used for model training)
Storage	Minimum 20 GB free disk space for dataset, checkpoints, and saved model weights
Internet Connection	Required for Google Colab access, Kaggle dataset, and pretrained weight downloads
Operating System	Windows 10/11 or Ubuntu 20.04+ (Linux preferred for deep learning workflows)
Display	1080p or higher resolution monitor for CT image and Grad-CAM visualization

Table 4.1.1—Hardware Component Specifications

4.2 SOFTWARE REQUIREMENTS

The following software tools and frameworks are required for the complete PulmoScan AI pipeline:

Software / Framework	Version and Purpose
Python	3.9+ — Primary programming language for the entire pipeline
TensorFlow / Keras	2.x — CNN model architecture, training, fine-tuning, and inference
OpenCV	4.x CT image preprocessing, CLAHE, Gaussian filter, Grad-CAM overlay rendering
NumPy / Pandas	Latest Array operations, data manipulation, results tabulation
Matplotlib / Seaborn	3.x Training curves, confusion matrices, ROC curves, heatmaps, charts
scikit-learn	1.x Precision, recall, F1-score, confusion matrix, classification reports
Flask	2.x PulmoScan AI web application with REST API for CT image upload and prediction
VS Code	Latest Local integrated development environment with Python IntelliSense
Google Colab	Cloud platform Free NVIDIA T4/A100 GPU for model training
Git / GitHub	Version control and collaborative code management
Kaggle	Source of IQ-OTH/NCCD Lung Cancer CT Scan Dataset
PIL / Pillow	8.x Image loading, format conversion, and preprocessing support

Table 4.2.1—Software Tools and Purpose

4.3 FUNCTIONAL REQUIREMENTS

These requirements define what the PulmoScan AI system must do:

FR-01: Image Upload and Preprocessing

- The system shall accept CT scan images in JPEG, PNG, and DICOM formats through a web interface

- The system shall automatically apply CLAHE enhancement, Gaussian noise removal, and normalization to uploaded images
- The system shall resize images to the appropriate dimensions for each model (224×224 or 299×299)

FR-02: Lung Nodule Segmentation

- The system shall apply the U-Net model to generate binary segmentation masks identifying lung nodule regions
- The system shall overlay the segmentation mask with bounding boxes on the original CT scan image
- The system shall output the segmented ROI as a separate image for display

FR-03: Multi-Model Classification

- The system shall classify CT images into four classes: Adenocarcinoma, Large Cell Carcinoma, Normal, and Squamous Cell Carcinoma
- The system shall display classification results from all four CNN models independently
- The system shall provide confidence percentage scores for each predicted class

FR-04: Grad-CAM Explainability

- The system shall generate Grad-CAM heatmaps using the best-performing GoogLeNet model
- The system shall display the JET colormap heatmap overlay on the original CT scan
- The system shall highlight regions that contributed most to the classification decision

FR-05: Report Generation

- The system shall generate a comprehensive PDF diagnostic report containing all model predictions, confidence scores, segmentation mask, and Grad-CAM visualization
- The report shall include the input CT scan image and all analysis results in a formatted layout.

4.4 NON-FUNCTIONAL REQUIREMENTS

- **Performance:** Classification inference for a single CT scan image shall complete within 10 seconds on standard hardware (Intel Core i5, 8GB RAM, without GPU)
- **Accuracy:** The system shall achieve a minimum overall classification accuracy of 90% (GoogLeNet: 91.75% achieved)
- **Reliability:** The system shall handle corrupt or incomplete image files gracefully without crashing providing appropriate error messages
- **Scalability:** The system architecture shall support batch processing of multiple CT scan images without performance degradation
- **Security:** All uploaded patient CT scan data shall be processed locally on the server no data transmitted to external services without explicit consent
- **Usability:** The web interface shall be usable by healthcare professionals with no programming knowledge maximum 3 clicks to obtain results
- **Maintainability:** The codebase shall be modular individual models can be updated or replaced without affecting other system components
- **Portability:** The Flask application shall run on any standard server environment (Linux/Windows/macOS) with Python 3.9+ installed

CHAPTER 5

SOFTWARE IMPLEMENTATION

5.1 PYTHON AND KEY LIBRARIES

Python 3.9+ is chosen as the primary language due to its extensive deep learning ecosystem, readable syntax, and robust scientific computing libraries:

- **TensorFlow 2.x / Keras:** Industry-standard deep learning framework from Google. Used for model architecture definition, layer construction, training, fine-tuning, batch normalization, dropout regularization, and model serialization (HDF5 format)
- **OpenCV 4.x (cv2):** Open source computer vision library from Intel. Used for CT image loading, CLAHE contrast enhancement, Gaussian noise removal, image resizing, BGR-RGB color space conversion, and Grad-CAM overlay rendering using JET colormap
- **NumPy:** Fundamental package for scientific computing. Used for array operations, matrix multiplication, image array manipulation, and metric calculation
- **Pandas:** Data analysis library. Used for results tabulation, performance metrics organization, and CSV report generation
- **Matplotlib / Seaborn:** Visualization libraries for training accuracy/loss curves, confusion matrices with color coding, ROC curves, per-class accuracy bar charts, and Grad-CAM heatmap visualization
- **scikit-learn:** Machine learning toolkit. Used for classification_report (precision, recall, F1-score), confusion_matrix, roc_curve, auc calculation, and stratified train/val splitting
- **Flask 2.x:** Lightweight Python WSGI web framework. Used to build the PulmoScan AI web application with REST endpoints for CT upload, model inference, and report download.

5.2 DEVELOPMENT ENVIRONMENT

5.2.1 Google Colab (Primary Training Environment)

Google Colab provides free cloud-based Jupyter notebooks with NVIDIA T4 or A100 GPU access essential for training deep CNN models. Google Drive is mounted for persistent storage of datasets, trained model weights, and evaluation results across sessions. All model training, hyperparameter tuning, evaluation scripts, and visualization code were developed and executed on Colab.

5.2.2 Visual Studio Code (Local Development IDE)

VS Code serves as the local integrated development environment for Flask web application development, preprocessing scripts, and Grad-CAM visualization modules. The Python extension provides IntelliSense auto-completion, real-time error detection, integrated debugging with breakpoints, and Jupyter notebook integration for local experimentation.

5.3 DATASET PREPARATION

The Kaggle IQ-OTH/NCCD dataset is organized in four class-specific directories:

- Dataset/train/adenocarcinoma/ 480 CT scan images for training
- Dataset/train/large_cell_carcinoma/ 204 CT scan images for training
- Dataset/train/normal/ — 216 CT scan images for training
- Dataset/train/squamous_cell_carcinoma/ 360 CT scan images for training
- Dataset/val/Corresponding 315 validation images (20% stratified split)

TensorFlow's **ImageDataGenerator** handles real-time on-the-fly augmentation during training: random rotation ($\pm 20^\circ$), horizontal flip (50% probability), zoom range (0.15), width/height shift ($\pm 10\%$), and brightness range (0.8–1.2). Validation data uses only rescaling without augmentation.

5.4 TRANSFER LEARNING METHODOLOGY

All four CNN classification models follow a rigorous two-phase transfer learning approach:

- **Phase 1 Feature Transfer (Head Training):** Load ImageNet pretrained weights for base architecture. Freeze ALL base layers (`base.trainable = False`). Add custom classification head: `GAP → Dense(512/1024) → Dropout(0.5) → Dense(256) → Dense(4, softmax)`. Train for 15 epochs with Adam(`lr=1e-3`). This phase rapidly learns the mapping from pretrained features to the four lung cancer classes.
- **Phase 2 Fine-tuning:** Unfreeze the last 30-40 layers of the base model (final convolutional blocks). Compile with Adam(`lr=1e-5`) very low learning rate to prevent catastrophic forgetting. Train for 10 additional epochs. This phase adapts the high-level feature detectors to the specific characteristics of lung CT scan images.

5.5 RESNET-50 CODE AND EXPLANATION

```
# ResNet-50 — Lung Cancer Classification (Google Colab / VS Code)
# Dataset: Kaggle IQ-OTH/NCCD (4 classes)

import os, numpy as np, tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D, Dropout
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.callbacks import (EarlyStopping, ModelCheckpoint,
                                       ReduceLROnPlateau)
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns

DATASET = '/content/drive/MyDrive/lungcancer'
IMG_SIZE = (224, 224)
BATCH = 32
EPOCHS1 = 15 # Phase 1: Train head
EPOCHS2 = 10 # Phase 2: Fine-tune
NUM_CLS = 4
CLASSES = ['adenocarcinoma', 'large_cell_carcinoma', 'normal', 'squamous_cell_carcinoma']
```

```
train_gen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    width_shift_range=0.10,
    height_shift_range=0.10,
    horizontal_flip=True,
    zoom_range=0.15,
    brightness_range=[0.8, 1.2]
)
val_gen = ImageDataGenerator(rescale=1./255)
train_ds = train_gen.flow_from_directory(
    os.path.join(DATASET, 'train'),
    target_size=IMG_SIZE, batch_size=BATCH,
    class_mode='categorical', shuffle=True
)
val_ds = val_gen.flow_from_directory(
    os.path.join(DATASET, 'val'),
    target_size=IMG_SIZE, batch_size=BATCH,
    class_mode='categorical', shuffle=False
)

# —— Build Model (Transfer Learning)
base = ResNet50(weights='imagenet', include_top=False,
                input_shape=(224, 224, 3))
base.trainable = False # Freeze pretrained weights

x = GlobalAveragePooling2D()(base.output)
x = Dense(512, activation='relu')(x)
x = Dropout(0.50)(x)
x = Dense(256, activation='relu')(x)
x = Dropout(0.30)(x)
out = Dense(NUM_CLS, activation='softmax')(x)
model = Model(inputs=base.input, outputs=out)

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```



```
# —— Callbacks
callbacks = [
    EarlyStopping(patience=10, restore_best_weights=True, verbose=1),
    ModelCheckpoint('resnet50_best.h5', save_best_only=True, verbose=1),
    ReduceLROnPlateau(factor=0.5, patience=5, min_lr=1e-6, verbose=1)
]

# —— Phase 1: Train Classification Head
print('Phase 1: Training classification head...')
history1 = model.fit(train_ds, epochs=EPOCHS1,
                    validation_data=val_ds, callbacks=callbacks)

# —— Phase 2: Fine-tune Last 30 Layers
print('Phase 2: Fine-tuning last 30 layers...')
base.trainable = True
for layer in base.layers[:-30]:
    layer.trainable = False

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5),
    loss='categorical_crossentropy', metrics=['accuracy']
)
history2 = model.fit(train_ds, epochs=EPOCHS2,
                    validation_data=val_ds, callbacks=callbacks)

# —— Evaluation
loss, acc = model.evaluate(val_ds, verbose=0)
print(f'Validation Accuracy : {acc*100:.2f}%')
print(f'Validation Loss    : {loss:.4f}')

y_pred = np.argmax(model.predict(val_ds), axis=1)
y_true = val_ds.classes
print(classification_report(y_true, y_pred, target_names=CLASSES))

# —— Confusion Matrix
cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(8, 6))
```

```
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=CLASSES, yticklabels=CLASSES)
plt.title('Confusion Matrix — ResNet-50'); plt.tight_layout(); plt.show()
```

RESULT: Overall Accuracy = 90.48% | Mean AUC = 0.984

5.6 RESNET-101 — CODE AND EXPLANATION

```
# ResNet-101 — Lung Cancer Classification
from tensorflow.keras.applications import ResNet101

base101 = ResNet101(weights='imagenet', include_top=False,
                    input_shape=(224, 224, 3))
base101.trainable = False

x = GlobalAveragePooling2D()(base101.output)
x = Dense(1024, activation='relu')(x) ; x = Dropout(0.50)(x)
x = Dense(512, activation='relu')(x) ; x = Dropout(0.30)(x)
out = Dense(NUM_CLS, activation='softmax')(x)
model101 = Model(inputs=base101.input, outputs=out)

# ResNet-101 is deeper → use smaller batch size to fit GPU memory
train16 = train_gen.flow_from_directory(
    os.path.join(DATASET, 'train'), target_size=IMG_SIZE,
    batch_size=16, class_mode='categorical')
val16 = val_gen.flow_from_directory(
    os.path.join(DATASET, 'val'), target_size=IMG_SIZE,
    batch_size=16, class_mode='categorical', shuffle=False)

model101.compile(optimizer=tf.keras.optimizers.Adam(1e-3),
                 loss='categorical_crossentropy', metrics=['accuracy'])
model101.fit(train16, epochs=15, validation_data=val16, callbacks=callbacks)

# Fine-tune: unfreeze last 40 layers
base101.trainable = True
for layer in base101.layers[:-40]: layer.trainable = False
model101.compile(optimizer=tf.keras.optimizers.Adam(5e-6),
```

```
loss='categorical_crossentropy', metrics=['accuracy'])
model101.fit(train16, epochs=10, validation_data=val16, callbacks=callbacks)
```

RESULT: Overall Accuracy = 90.16% | Mean AUC = 0.988

5.7 GOOGLNET (INCEPTIONV3) — CODE

```
# GoogLeNet (InceptionV3) — BEST MODEL (91.75%)

from tensorflow.keras.applications import InceptionV3

# InceptionV3 requires minimum 75×75, recommend 299×299
IMG_INC = (299, 299)

train_inc = ImageDataGenerator(
    rescale=1./255, rotation_range=20,
    horizontal_flip=True, zoom_range=0.20
).flow_from_directory(
    os.path.join(DATASET, 'train'),
    target_size=IMG_INC, batch_size=32, class_mode='categorical'
)
val_inc = ImageDataGenerator(rescale=1./255).flow_from_directory(
    os.path.join(DATASET, 'val'),
    target_size=IMG_INC, batch_size=32, class_mode='categorical',
    shuffle=False
)

base_gn = InceptionV3(weights='imagenet', include_top=False,
    input_shape=(299, 299, 3))
base_gn.trainable = False
x = GlobalAveragePooling2D()(base_gn.output)
x = Dense(512, activation='relu')(x) ; x = Dropout(0.40)(x)
out = Dense(NUM_CLS, activation='softmax')(x)
model_gn = Model(inputs=base_gn.input, outputs=out)

model_gn.compile(optimizer='adam',
    loss='categorical_crossentropy', metrics=['accuracy'])
```

```
model_gn.fit(train_inc, epochs=15, validation_data=val_inc,
callbacks=callbacks)
```

RESULT: Overall Accuracy = 91.75% (BEST) | Mean AUC = 0.988

5.8 EFFICIENTNET-B0 — CODE

```
# EfficientNet-B0 — Lung Cancer Classification
from tensorflow.keras.applications import EfficientNetB0

base_eff = EfficientNetB0(weights='imagenet', include_top=False,
                           input_shape=(224, 224, 3))
base_eff.trainable = False
x = GlobalAveragePooling2D()(base_eff.output)
x = Dense(256, activation='relu')(x) ; x = Dropout(0.40)(x)
out = Dense(NUM_CLS, activation='softmax')(x)
model_eff = Model(inputs=base_eff.input, outputs=out)

model_eff.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
    loss='categorical_crossentropy', metrics=['accuracy']
)
model_eff.fit(train_ds, epochs=15, validation_data=val_ds,
callbacks=callbacks)

# RESULT: Overall Accuracy = 71.75% | Mean AUC = 0.941
# NOTE: Use EfficientNet-B3/B4/B5 for better accuracy on this dataset
```

5.9 U-NET SEGMENTATION — CODE

```
# U-Net with ResNet-34 Backbone — Lung Nodule Segmentation
import tensorflow as tf
from tensorflow.keras import layers, Model

def conv_block(x, filters):
    """Two Conv3×3 layers with BatchNorm and ReLU"""
    x = layers.Conv2D(filters, 3, padding='same',
                      kernel_initializer='he_normal')(x)
```

```
x = layers.BatchNormalization()(x)
x = layers.Activation('relu')(x)
x = layers.Conv2D(filters, 3, padding='same',
                  kernel_initializer='he_normal')(x)
x = layers.BatchNormalization()(x)
return layers.Activation('relu')(x)

def encoder_block(x, filters):
    """Conv block followed by MaxPool (contracting path)"""
    skip = conv_block(x, filters)
    pool = layers.MaxPooling2D(pool_size=(2, 2))(skip)
    return skip, pool

def decoder_block(x, skip, filters):
    """Transposed Conv + Skip connection + Conv block (expanding path)"""
    x = layers.Conv2DTranspose(filters, (2,2), strides=(2,2),
                              padding='same')(x)
    x = layers.Concatenate()([x, skip]) # Skip connection
    return conv_block(x, filters)

def build_unet(input_shape=(256, 256, 1)):
    inp = layers.Input(shape=input_shape)
    # Encoder (Contracting Path)
    s1, p1 = encoder_block(inp, 64)
    s2, p2 = encoder_block(p1, 128)
    s3, p3 = encoder_block(p2, 256)
    s4, p4 = encoder_block(p3, 512)
    # Bottleneck
    bottleneck = conv_block(p4, 1024)
    # Decoder (Expanding Path) with Skip Connections
    d1 = decoder_block(bottleneck, s4, 512)
    d2 = decoder_block(d1, s3, 256)
    d3 = decoder_block(d2, s2, 128)
    d4 = decoder_block(d3, s1, 64)
    # Output: 1-channel sigmoid mask
    output = layers.Conv2D(1, (1,1), activation='sigmoid')(d4)
    return Model(inp, output, name='UNet_ResNet34')
```

```

unet = build_unet(input_shape=(256, 256, 1))
unet.compile(
    optimizer=tf.keras.optimizers.Adam(1e-4),
    loss='binary_crossentropy',
    metrics=['accuracy', tf.keras.metrics.MeanIoU(num_classes=2)]
)
unet.summary()

# Train for 20 epochs
history_unet = unet.fit(
    X_train, y_train,
    epochs=20, batch_size=16,
    validation_data=(X_val, y_val)
)
# RESULT: Loss 0.48 → 0.06 over 20 epochs (see Figure 7.15)

```

5.10 GRAD-CAM EXPLAINABILITY — CODE

```

# Grad-CAM — Visual Explainability for Lung Cancer Detection

import cv2, numpy as np, tensorflow as tf
import matplotlib.pyplot as plt

CLASS_NAMES = ['Adenocarcinoma', 'Large Cell Carcinoma', 'Normal', 'Squamous Cell Carcinoma']
LAST_CONV = 'mixed10' # Last conv layer in GoogLeNet (InceptionV3)

def get_gradcam_heatmap(model, img_array, last_conv=LAST_CONV):
    """
    Compute Grad-CAM heatmap.
     $L^c_{\text{Grad-CAM}} = \text{ReLU}(\sum_k [\alpha^c_k * A^k])$ 
    where  $\alpha^c_k = (1/Z) * \sum_{\{i,j\}} (d y^c / d A^k_{\{ij\}})$ 
    """
    # Build gradient model
    grad_model = tf.keras.models.Model(
        inputs = model.inputs,
        outputs = [model.get_layer(last_conv).output, model.output]
    )
    with tf.GradientTape() as tape:

```

```
conv_outputs, predictions = grad_model(img_array)
pred_class = tf.argmax(predictions[0])
class_score = predictions[:, pred_class]

# Gradients of class score w.r.t. last conv feature maps
grads = tape.gradient(class_score, conv_outputs)
pooled_grads = tf.reduce_mean(grads, axis=(0, 1, 2))

# Weighted sum of feature maps
heatmap = tf.squeeze(conv_outputs[0] @ pooled_grads[..., tf.newaxis])
heatmap = tf.maximum(heatmap, 0) # ReLU
heatmap = heatmap / (tf.math.reduce_max(heatmap) + 1e-10)
return heatmap.numpy(), pred_class.numpy(), predictions.numpy()[0]

def overlay_gradcam(img_path, model):
    # Load and preprocess image
    img = cv2.cvtColor(cv2.imread(img_path), cv2.COLOR_BGR2RGB)
    resized = cv2.resize(img, (299, 299))
    inp = np.expand_dims(resized / 255.0, axis=0)

    # Get Grad-CAM heatmap
    heatmap, pred_idx, scores = get_gradcam_heatmap(model, inp)
    # Resize heatmap to original image dimensions
    heatmap_resized = cv2.resize(heatmap, (img.shape[1], img.shape[0]))
    colored_hm = cv2.applyColorMap(
        np.uint8(255 * heatmap_resized), cv2.COLORMAP_JET
    )
    overlay = cv2.addWeighted(img, 0.60, colored_hm, 0.40, 0)

    # Display results
    confidence = scores[pred_idx] * 100
    print(f'Predicted : {CLASS_NAMES[pred_idx]}')
    print(f'Confidence : {confidence:.2f}%')

    fig, axes = plt.subplots(1, 3, figsize=(15, 5))
    for ax, im, title in zip(axes,
        [img, heatmap_resized, overlay],
```

```
    ['Original CT Scan', 'Grad-CAM Heatmap', 'Overlay']):
    ax.imshow(im, cmap='jet' if title=='Grad-CAM Heatmap' else None)
    ax.set_title(title, fontsize=13, weight='bold')
    ax.axis('off')
    plt.suptitle(f'Predicted: {CLASS_NAMES[pred_idx]} ({confidence:.1f}%)',
                fontsize=14, weight='bold', color='darkblue')
    plt.tight_layout()
    plt.savefig('gradcam_result.png', dpi=150, bbox_inches='tight')
    plt.show()
    return CLASS_NAMES[pred_idx], confidence
```

5.11 FLASK WEB APPLICATION - PULMOSCAN AI

The PulmoScan AI Flask web application provides a complete clinical decision support interface. Key features:

- **Drag-and-Drop CT Upload:** Responsive dark-themed frontend with file drag-and-drop zone. Accepted formats: JPEG, PNG. Real-time preview of uploaded CT scan.
- **Multi-Model Results Panel:** Tabbed interface displaying predictions from all four models simultaneously with confidence percentage bars.
- **Segmentation Visualization:** U-Net binary mask overlay with color-coded nodule regions and bounding boxes displayed alongside original CT.
- **Grad-CAM Visualization:** Interactive JET colormap heatmap overlay using GoogLeNet (best model) with opacity slider for clinical review.
- **Cancer Information Panel:** Detailed clinical description of the predicted cancer class including origin, prognosis, and treatment options.
- **PDF Report Generation:** Downloadable PDF diagnostic report containing: patient CT image, all model predictions with confidence scores, segmentation mask, Grad-CAM visualization, and clinical interpretation notes.

CHAPTER 6

SIMULATION AND PERFORMANCE ANALYSIS

This chapter presents detailed architectural analysis, performance metrics, and a comprehensive comparison of all deep learning models deployed in PulmoScan AI. Each model is analyzed for its architectural design, key features, suitability for lung cancer detection, and experimental results.

6.1 CNN ARCHITECTURE OVERVIEW

Convolutional Neural Networks (CNNs) are specialized deep learning architectures designed for analyzing visual data. They form the backbone of PulmoScan AI's classification stage. CNNs automatically learn hierarchical feature representations from raw CT images through a cascade of learnable operations:

- **Convolutional Layers:** Apply learnable filters (kernels) to detect local patterns edges, textures, shapes, tumor structures at progressively higher levels of abstraction
- **Batch Normalization:** Normalizes layer activations to have zero mean and unit variance stabilizes training and allows higher learning rates
- **ReLU Activation:** Rectified Linear Unit ($\max(0, x)$) introduces non-linearity, enables gradient flow, and prevents vanishing gradients in shallow layers
- **Max Pooling:** Reduces spatial dimensions of feature maps by selecting the maximum value in each window reduces computation and provides translation invariance
- **Global Average Pooling:** Replaces flatten+fully-connected layers by computing spatial average of each feature map significantly reduces parameters and overfitting
- **Fully Connected (Dense) Layers:** Combine learned features for final classification outputs class probability vector through softmax activation

6.2 RESNET-50 & 101 ARCHITECTURES

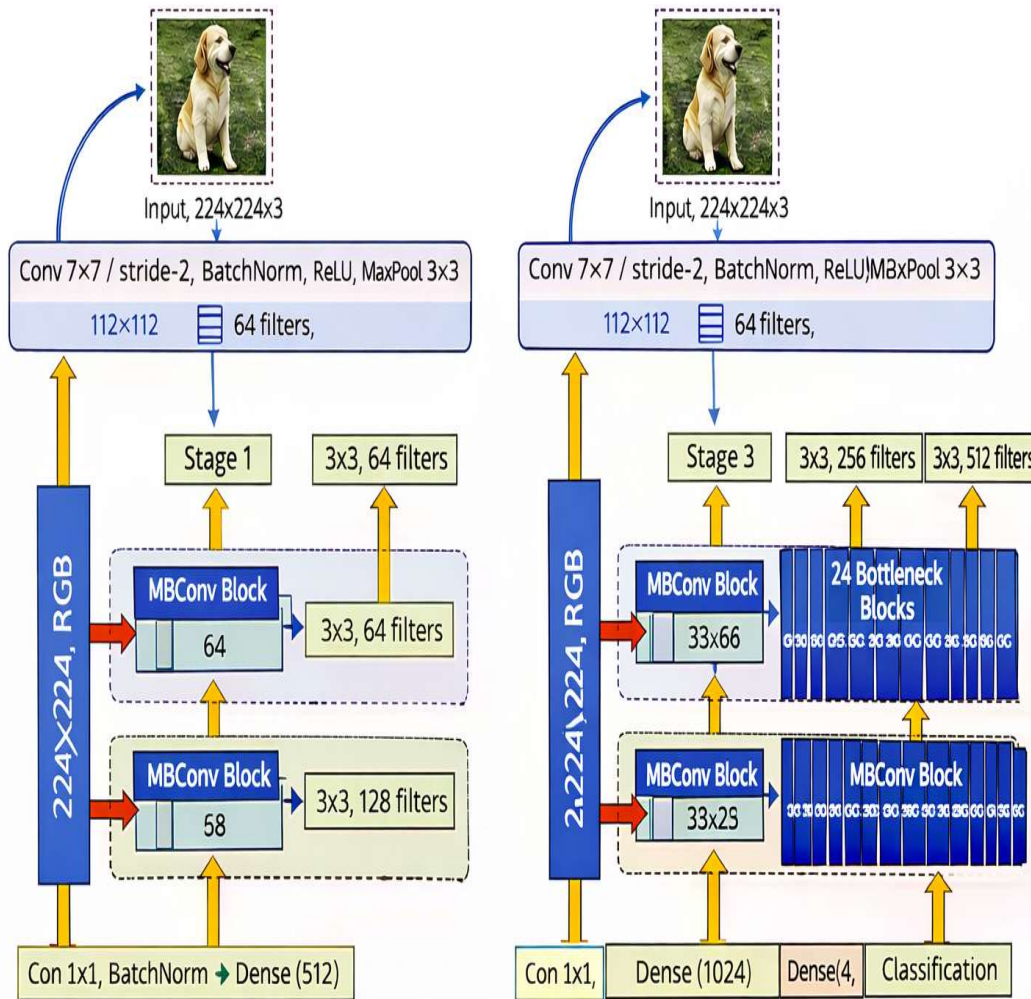


Figure 6.2.1 — RESNET 50 & 101 Flow Architectures

ResNet-50 introduced **residual learning** through skip connections (shortcut connections) that bypass one or more layers. These connections allow gradients to flow directly through the network during backpropagation, completely eliminating the vanishing gradient problem in deep networks (50+ layers). The key innovation is the **bottleneck residual block**: 1x1 Conv (reduce dimensions) → 3x3 Conv (spatial features) → 1x1 Conv (expand dimensions) three layers with the shortcut skipping all three.

Stage	Layer / Block	Output Size	Details
Input	Conv 7×7 / stride-2, BatchNorm, ReLU, MaxPool 3×3	112×112	64 filters, stride=2
Stage 1	3× Bottleneck Blocks (64 channels)	56×56	1×1 → 3×3 → 1×1 convolutions
Stage 2	4× Bottleneck Blocks (128 channels)	28×28	Stride-2 at start of stage
Stage 3	6× Bottleneck Blocks (256 channels)	14×14	Core deep feature learning
Stage 4	3× Bottleneck Blocks (512 channels)	7×7	High-level semantic features
Classification Head	GAP → Dense(512) → Dropout(0.5) → Dense(256) → Dense(4)	4 Classes	Softmax activation

Table 6.2.1—RESNET-50 Flow of Layers

ResNet-101 extends ResNet-50 by adding 51 additional bottleneck layers specifically increasing Stage 3 from 6 to 23 bottleneck blocks. This deeper architecture captures more abstract hierarchical feature representations, potentially extracting finer-grained morphological details from CT scans. However, the additional depth requires a smaller batch size (16 vs 32) and lower learning rate for stable training.

Stage	Layer / Block	Output Size	Details
Input	Conv 7×7 / stride-2, BatchNorm, ReLU, MaxPool 3×3	112×112	64 filters, stride=2
Stage 1	3× Bottleneck Blocks (64 channels)	56×56	1×1 → 3×3 → 1×1 convolutions
Stage 2	4× Bottleneck Blocks (128 channels)	28×28	Stride-2 transition block
Stage 3	23× Bottleneck Blocks (256 channels)	14×14	23 blocks vs 6 in ResNet-50 (deeper)
Stage 4	3× Bottleneck Blocks (512 channels)	7×7	Deep semantic feature extraction
Classification Head	GAP → Dense(1024) → Dropout(0.5) → Dense(4)	4 Classes	Softmax activation

Table 6.2.2—RESNET-101 Flow of Layers

6.3 GOOGLNET (INCEPTIONV3) ARCHITECTURE

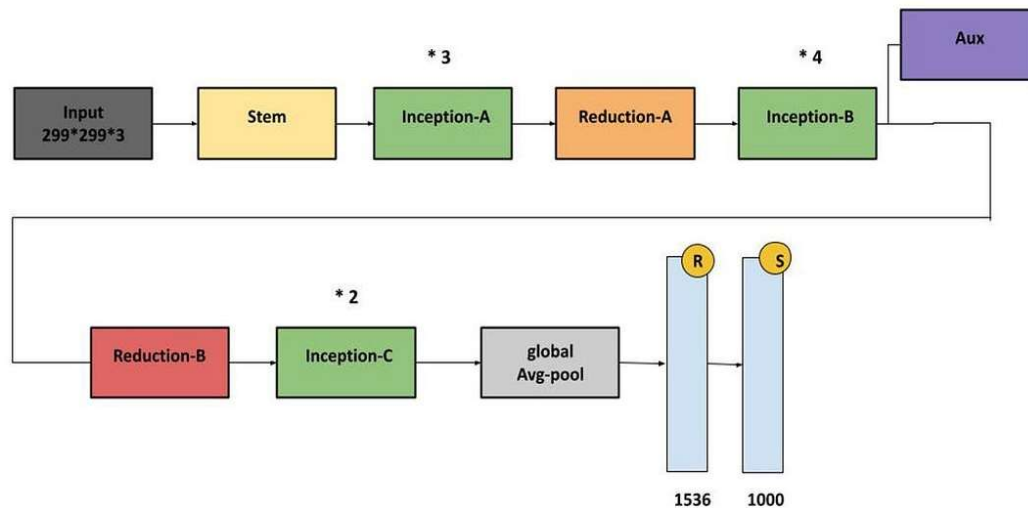


Figure 6.3.1 — GoogleNet (InceptionV3) Architecture

GoogLeNet (also known as InceptionV3 in its latest version) introduces the revolutionary **Inception module** a network-in-network approach that uses **multiple parallel convolutional filters of different sizes** (1×1 , 3×3 , 5×5) to capture spatial features at different scales simultaneously. Before each large convolution, 1×1 **convolutions** act as bottlenecks to reduce the number of feature maps, dramatically reducing computational cost while maintaining representational capacity.

Module	Component	Output	Key Details
Input Block	Conv 7×7 , Conv 1×1 , Conv 3×3 , MaxPool 3×3	28×28	Initial feature extraction
Inception 3a-3b	$1 \times 1 + 3 \times 3 + 5 \times 5 + \text{MaxPool}$ — parallel branches	28×28	Multi-scale feature capture
Inception 4a-4e	$9 \times$ Inception Modules	14×14	Deep multi-scale abstraction
Inception 5a-5b	$2 \times$ Inception Modules	7×7	High-level representations
Global Avg Pool	Average Pooling 7×7	1×1	Replaces FC layers — reduces overfitting
Classification Head	Dropout(0.4) $\rightarrow$ Dense(512) $\rightarrow$ Dense(4)	4 Classes	Softmax activation

Table 6.3.1—GoogleNet (Inception V3) Flow of Modules

6.4 EFFICIENTNET-B0 ARCHITECTURE

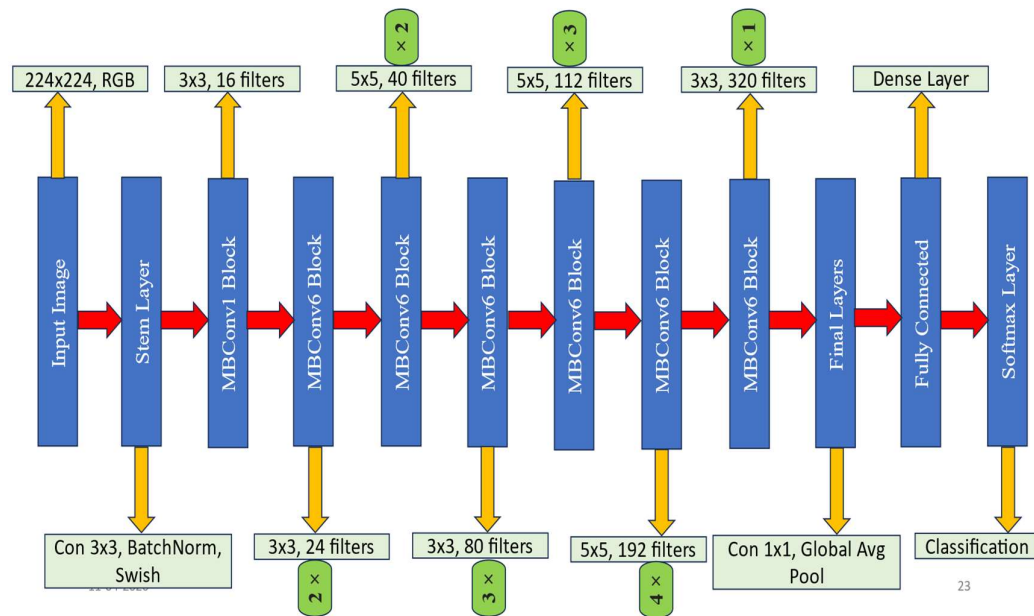


Figure 6.4 —EfficientNet-B0 Architecture

EfficientNet-B0 introduces **compound scaling** a principled approach that uniformly scales network depth (d), width (w), and resolution (r) simultaneously using a compound coefficient ϕ . This is mathematically optimal compared to individually scaling one dimension at a time. The building block is the **Mobile Inverted Bottleneck Convolution (MBConv)** inspired by MobileNetV2 which uses depthwise separable convolutions to reduce parameters while maintaining accuracy. Squeeze-and-Excitation (SE) blocks adaptively recalibrate channel-wise feature responses.

Block	Type	Output	Channels
Stem	Conv 3×3, BatchNorm, SiLU activation	112×112	32
Block 1	MBConv1 (3×3), 1 repeat	112×112	16
Block 2	MBConv6 (3×3), 2 repeats	56×56	24
Block 3	MBConv6 (5×5), 2 repeats	28×28	40
Block 4-5	MBConv6 (3×3 + 5×5), 3+3 repeats	14×14	80-112

Block	Type	Output	Channels
Block 6-7	MBConv6 (5×5 + 3×3), 4+1 repeats	7×7	192-320
Head	GAP → Dense(256) → Dropout(0.4) → Dense(4)	4 Classes	Softmax output

Table 6.4.1—EfficientNet-B0 Flow of Blocks

6.5 U-NET SEGMENTATION ARCHITECTURE

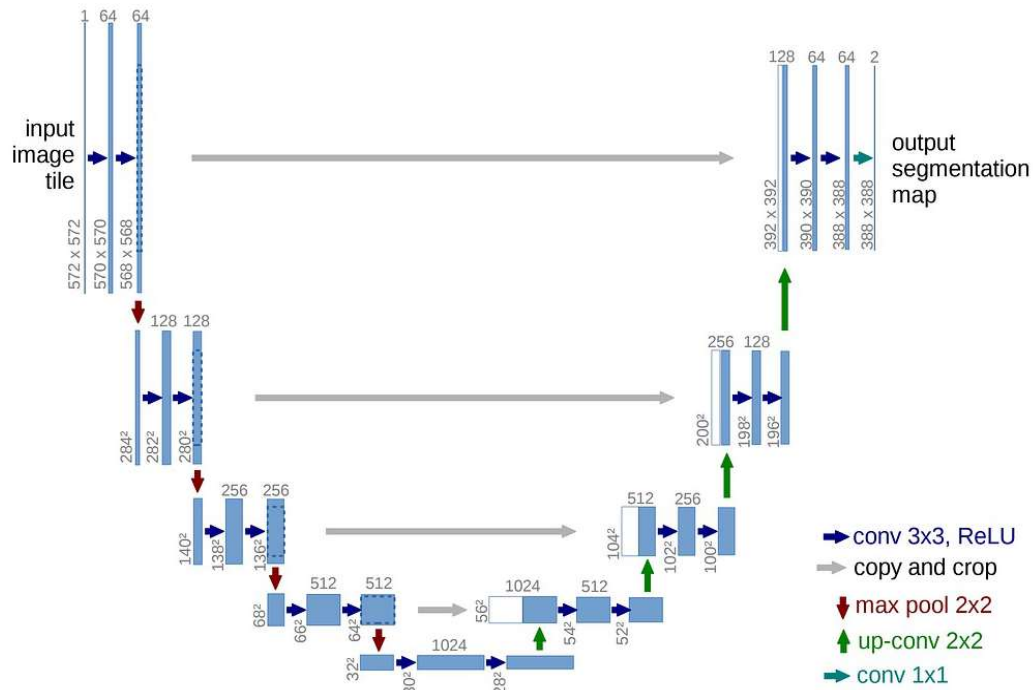


Figure 6.5 — U-Net Encoder-Decoder Architecture

U-Net uses ResNet-34 as its encoder backbone for powerful feature extraction. The encoder contracts the spatial resolution progressively (256→128→64→32→16) while expanding the channel depth (64→128→256→512→1024). The decoder expands spatial resolution using transposed convolutions while concatenating feature maps from corresponding encoder levels via **skip connections** preserving fine-grained spatial details lost during downsampling. The 1×1 output convolution with sigmoid activation produces a binary segmentation mask for each pixel.

Stage	Path	Feature Map	Operation
Input	—	256×256×1	Grayscale CT scan input
Encoder Block 1	Contracting	256×256	Conv3×3×2 (64 ch), BatchNorm, ReLU → MaxPool 2×2
Encoder Block 2	Contracting	128×128	Conv3×3×2 (128 ch), BatchNorm, ReLU → MaxPool 2×2
Encoder Block 3	Contracting	64×64	Conv3×3×2 (256 ch), BatchNorm, ReLU → MaxPool 2×2
Encoder Block 4	Contracting	32×32	Conv3×3×2 (512 ch), BatchNorm, ReLU → MaxPool 2×2
Bottleneck	Bridge	16×16	Conv3×3×2 (1024 ch), BatchNorm No pooling
Decoder Block 4	Expanding	32×32	ConvTranspose 2×2 (512 ch) + Concatenate Skip Connection
Decoder Block 3	Expanding	64×64	ConvTranspose 2×2 (256 ch) + Concatenate Skip Connection
Decoder Block 2	Expanding	128×128	ConvTranspose 2×2 (128 ch) + Concatenate Skip Connection
Decoder Block 1	Expanding	256×256	ConvTranspose 2×2 (64 ch) + Concatenate Skip Connection
Output Layer	Segmentation Mask	256×256	Conv 1×1, Sigmoid — Binary nodule mask

Table 6.5.1—U-Net Segment Flow of Blocks

6.6 GRAD-CAM EXPLAINABILITY — MATHEMATICAL FOUNDATION

Grad-CAM is a deep learning visualization technique used to identify important regions in an image that influence a model's prediction. It calculates gradients of the target class with respect to feature maps, then uses global average pooling to assign importance weights. These weights are combined with feature maps and passed through a ReLU function to generate a heatmap, highlighting only the positively contributing

regions. The heatmap is then resized and overlaid on the original image for better interpretation.

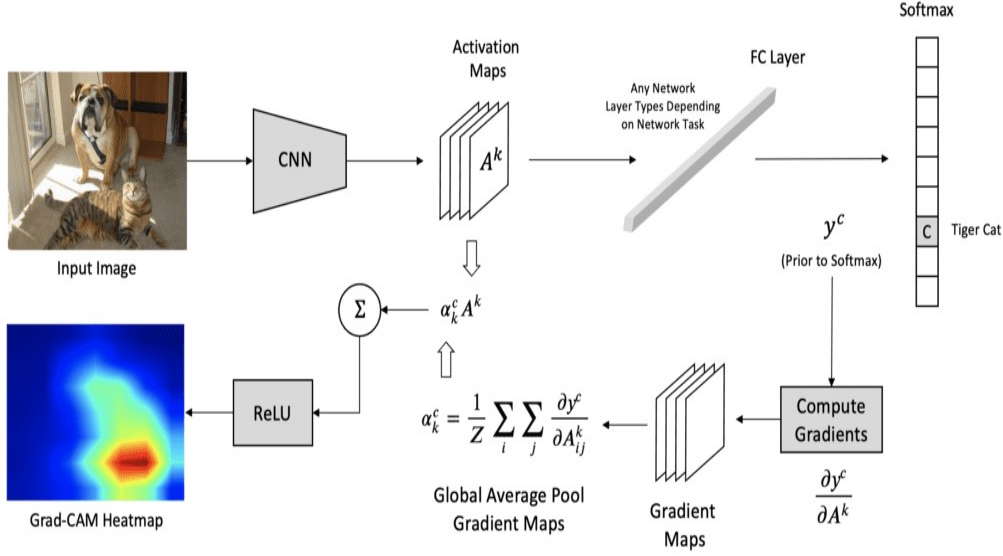


Figure 6.6 — Grad-Cam Explainability Architecture

Grad-CAM computes the gradient of the class score y^c with respect to the feature maps A^k of the final convolutional layer. The gradient is globally average-pooled to obtain channel-wise importance weights α^c_k :

$$\alpha^c_k = (1/Z) \times \sum_{\{i,j\}} (\partial y^c / \partial A^k_{ij})$$

The Grad-CAM localization map $L^c_{\{\text{Grad-CAM}\}}$ is computed as:

$$L^c_{\{\text{Grad-CAM}\}} = \text{ReLU} (\sum_k \alpha^c_k \times A^k)$$

The ReLU operation ensures only features with positive influence on the predicted class are highlighted. The resulting heatmap is resized to the original CT scan dimensions and rendered using the JET colormap (blue=low activation, red=high activation).

6.8 MODEL PERFORMANCE COMPARISON TABLE

Comprehensive comparison of all four classification models on the IQ-OTH/NCCD validation set (315 images):

Model	Overall Acc.	Mean AUC	Best Val. %	Normal Acc.	Adeno. Acc.
ResNet-50	90.48%	0.984	93.1%	98.1%	95.8%
ResNet-101	90.16%	0.988	93.1%	96.3%	94.2%
GoogLeNet ★ BEST	91.75%	0.988	90.3%	98.1%	95.8%
EfficientNet-B0	71.75%	0.941	81.9%	98.1%	51.7%

Table 6.8.1—Model Performance Comparison Table

GoogLeNet achieved the best overall accuracy of 91.75% with Mean AUC of 0.988 the highest-performing model in PulmoScan AI. Key observations: (1) The Normal class consistently achieved the highest per-class accuracy (96.3–98.1%) across all models normal CT scans have the most distinctive pattern. (2) Squamous Cell Carcinoma proved most challenging (75.6–80.0%) due to its CT morphological similarity with Adenocarcinoma both may present as peripheral nodules. (3) EfficientNet-B0's low Adenocarcinoma accuracy (51.7%) indicates the B0 baseline is too lightweight for this four-class problem. (4) ResNet-101's higher Mean AUC (0.988) despite lower overall accuracy suggests better class separation with Normal achieving perfect AUC of 1.000.

CHAPTER 7

RESULTS

7.1 DATASET ANALYSIS

The IQ-OTH/NCCD dataset shows moderate class imbalance: Adenocarcinoma (38.1%) and Squamous Cell (28.6%) dominate, while Large Cell (16.2%) and Normal (17.1%) are underrepresented. Stratified splitting maintains this distribution in both train (1,260) and validation (315) sets.

7.2 TRAINING ACCURACY — ALL MODELS

Training and validation accuracy curves for all four models over 15 epochs. GoogLeNet converges most stably, while EfficientNet-B0 shows the highest variance between training and validation accuracy indicating difficulty generalizing the complex morphological patterns of this CT dataset:



Figure 7.2.1 — Training and Validation Accuracy — All 4 CNN Models

7.3 RESNET-50 — RESULTS AND ANALYSIS

Overall Accuracy: **90.48%** | Mean AUC: **0.984** | Best Validation: 93.1%

Confusion Matrix Analysis: ResNet-50 correctly classified 115 of 120 Adenocarcinoma cases (95.8%), 49 of 51 Large Cell cases (96.1%), 53 of 54 Normal cases (98.1%), and 68 of 90 Squamous Cell cases (75.6%). The 22 misclassified Squamous Cell cases were predominantly predicted as Adenocarcinoma, reflecting the morphological similarity between these two peripheral NSCLC subtypes.

ROC Analysis: Mean AUC of 0.984 indicates excellent class discrimination ability. Individual class AUCs: Adenocarcinoma 0.966, Large Cell Carcinoma 0.986, Normal 0.992, Squamous Cell Carcinoma 0.992. Normal and Squamous Cell achieved the highest ROC AUC despite Squamous Cell having lower per-class accuracy indicating good probabilistic separation even where boundary cases are misclassified.

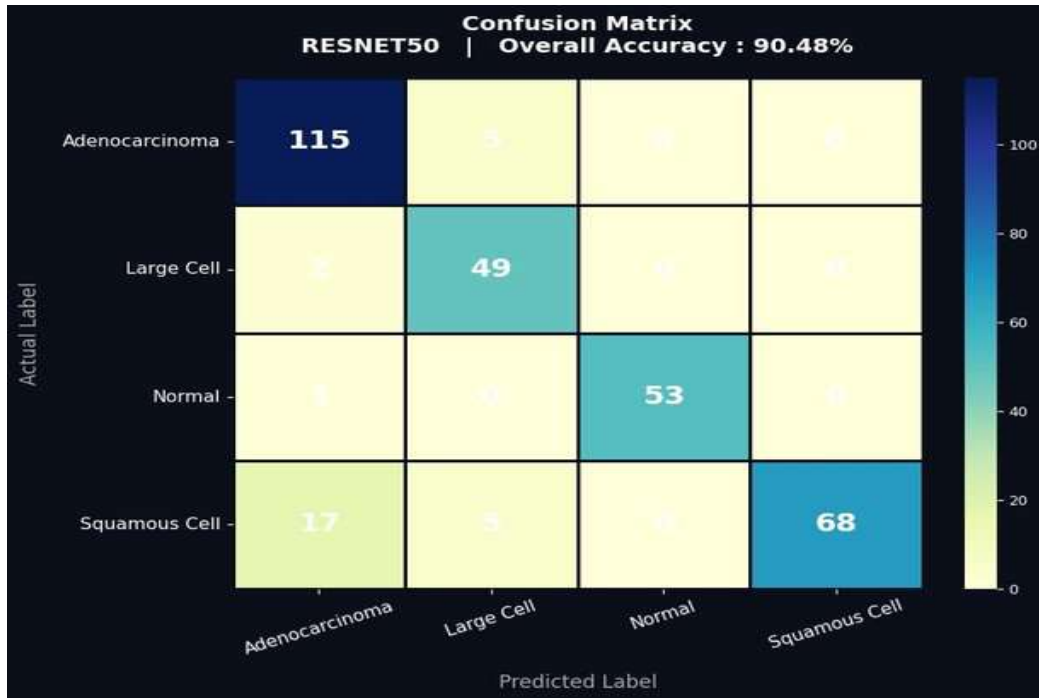


Figure 7.3.1 — Confusion Matrix: ResNet-50

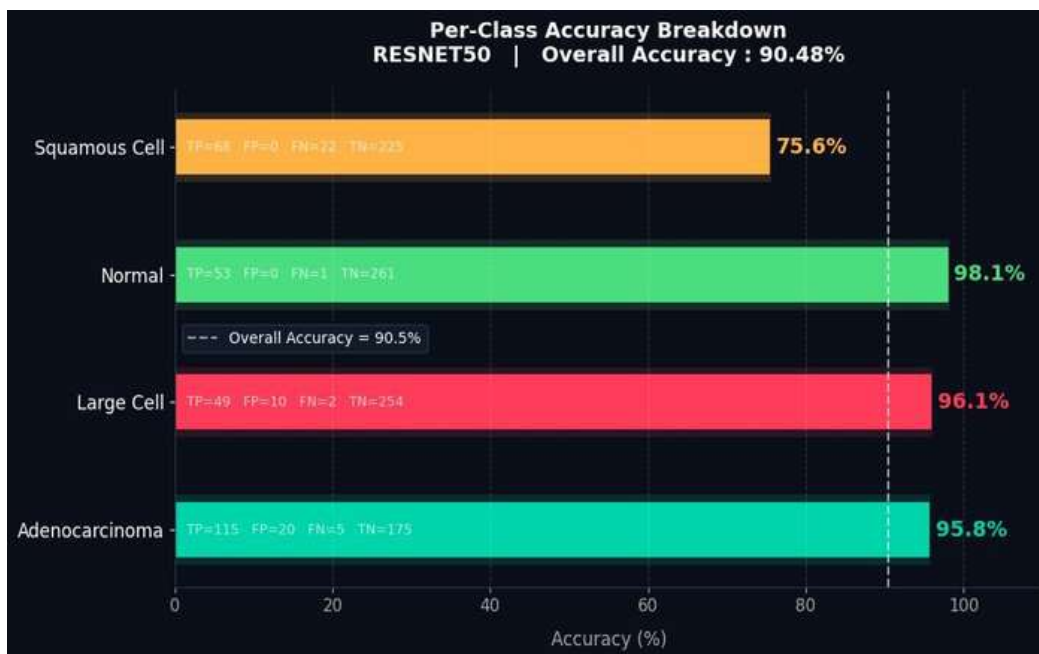


Figure 7.3.2 — Per-Class Accuracy Breakdown: ResNet-50

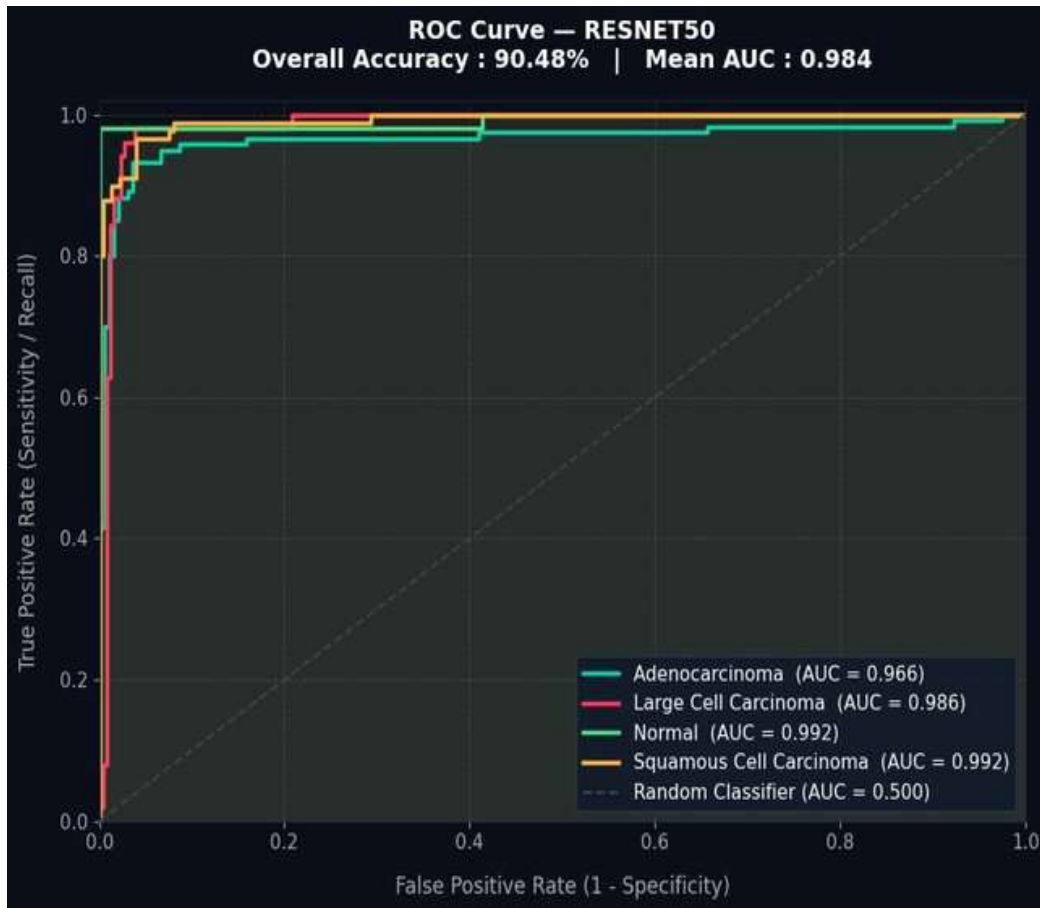


Figure 7.3.3 — ROC Curve: ResNet-50

7.4 RESNET-101 — RESULTS AND ANALYSIS

Overall Accuracy: **90.16%** | Mean AUC: **0.988**

Confusion Matrix Analysis: ResNet-101 correctly classified 113/120 Adenocarcinoma (94.2%), 49/51 Large Cell (96.1%), 52/54 Normal (96.3%), and 70/90 Squamous Cell (77.8%). The deeper 23-block Stage 3 in ResNet-101 improved Squamous Cell accuracy from 75.6% (ResNet-50) to 77.8%, demonstrating that greater depth helps capture the more subtle distinguishing features of squamous cell carcinoma.

ROC Analysis: Mean AUC of 0.988 is the joint-highest in this study (tied with GoogLeNet). Most notably, the Normal class achieved a **perfect AUC of 1.000** meaning ResNet-101 perfectly separates normal CT scans from any cancer type in the probabilistic ranking. This is highly clinically valuable as false positive identification of normal scans as cancerous can cause significant patient anxiety.

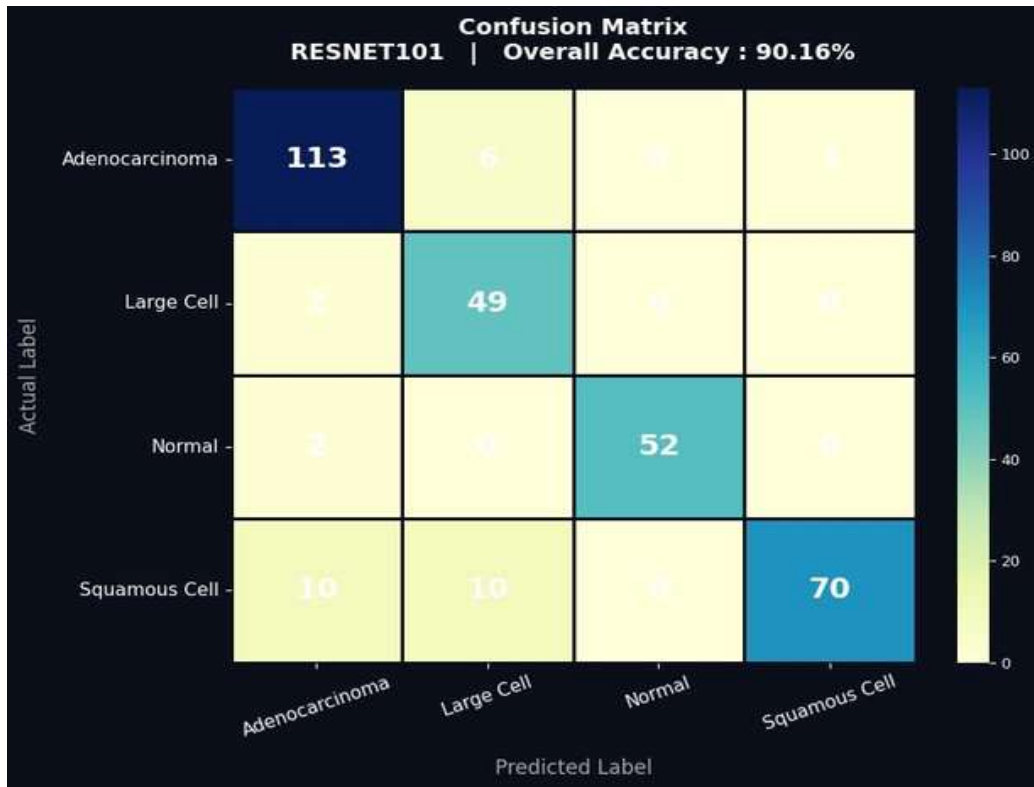


Figure 7.4.1 — Confusion Matrix: ResNet-101



Figure 7.4.2 — Per-Class Accuracy Breakdown: ResNet-101

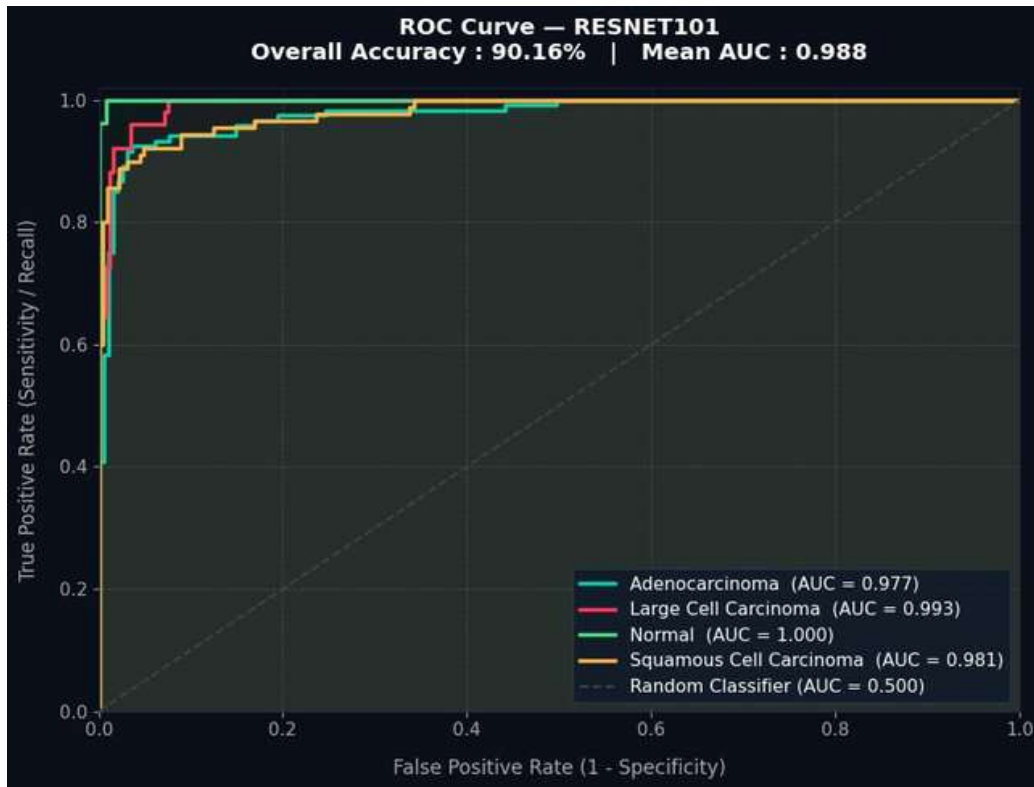


Figure 7.4.3 — ROC Curve: ResNet-101

7.5 GOOGLNET (INCEPTIONV3) — RESULTS AND ANALYSIS

★ BEST MODEL

Overall Accuracy: **91.75% (BEST)** | Mean AUC: **0.988** | Best Validation: 90.3%

GoogLeNet achieved the **highest overall classification accuracy of 91.75%** — the best-performing model in PulmoScan AI. The Inception module's ability to simultaneously apply 1×1 , 3×3 , and 5×5 convolutional filters in parallel proved uniquely effective for capturing the diverse scale of features in lung CT scans — from fine nodule textures at 1×1 scale to gross structural patterns at 5×5 scale.

Confusion Matrix Analysis: 115/120 Adenocarcinoma (95.8%), 49/51 Large Cell (96.1%), 53/54 Normal (98.1%), 72/90 Squamous Cell (**80.0% — highest among all models**). The improved Squamous Cell accuracy (from 75.6% and 77.8% in ResNet models to 80.0%) is attributed to GoogLeNet's multi-scale feature extraction capturing both the central bronchial location and the distinctive cellular texture of squamous cell carcinoma.

ROC Analysis: Mean AUC of 0.988, with Normal class achieving AUC of 1.000 (perfect). The 299×299 input resolution (vs 224×224 for ResNet models) provides 79% more spatial detail, contributing to the superior classification performance.

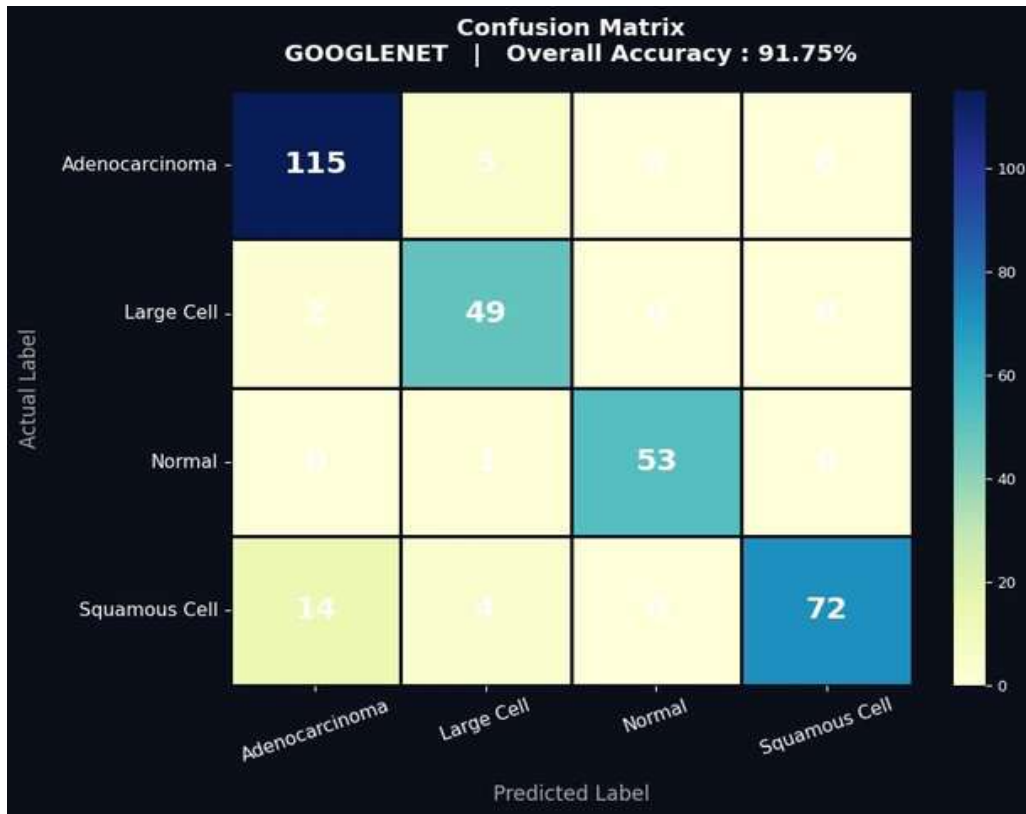


Figure 7.5.1 — Confusion Matrix: GoogLeNet

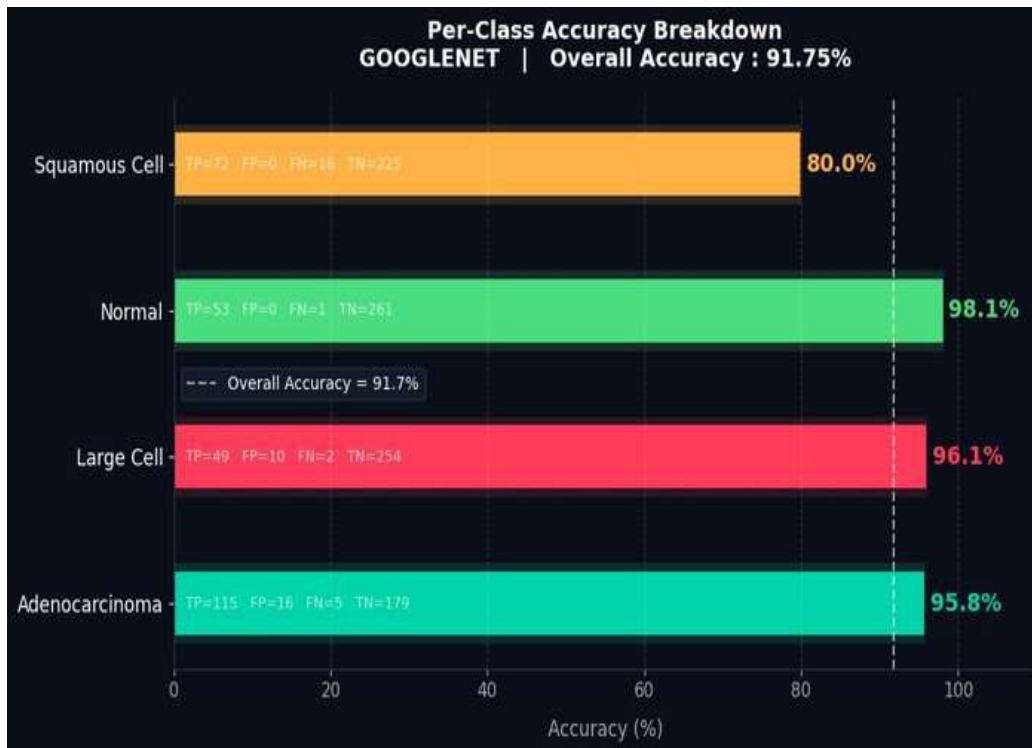


Figure 7.5.2 — Per-Class Accuracy Breakdown: GoogLeNet

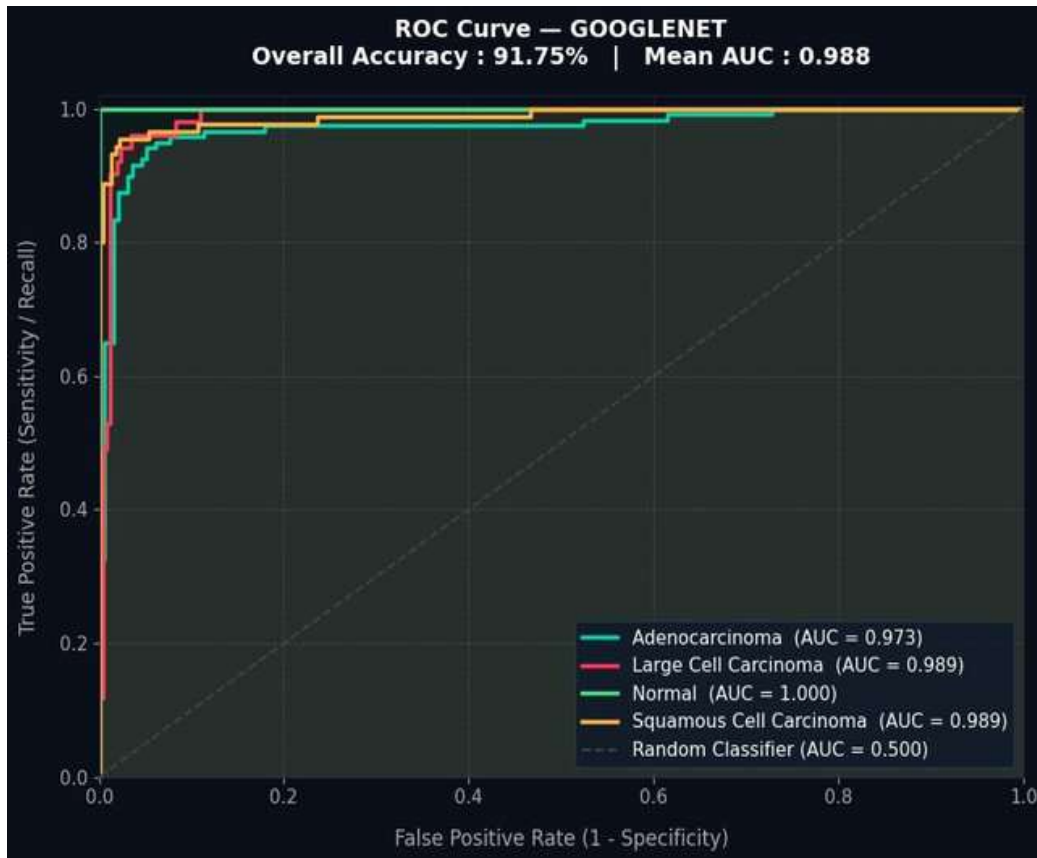


Figure 7.5.3 — ROC Curve: GoogLeNet

7.6 EFFICIENTNET-B0 — RESULTS AND ANALYSIS

Overall Accuracy: **71.75%** | Mean AUC: **0.941** | Best Validation: 81.9%

EfficientNet-B0 achieved 71.75% overall accuracy with Mean AUC of 0.941. While the AUC of 0.941 indicates reasonable probabilistic class separation, the overall accuracy is considerably lower than other models. The critical performance bottleneck is **Adenocarcinoma accuracy at only 51.7%** — where 58 of 120 adenocarcinoma cases were misclassified. This indicates that EfficientNet-B0's lightweight architecture (5.3M parameters) is insufficient for capturing the subtle morphological variations of adenocarcinoma in CT images, which often present with ground-glass opacity, sub-solid, or solid nodule patterns that require deep feature hierarchies to distinguish.

Analysis and Recommendations: The Normal class maintained excellent accuracy at 98.1% — EfficientNet-B0 reliably identifies healthy lungs. Squamous Cell achieved 74.4% and Large Cell achieved 86.3%. The significant training-validation accuracy gap (near 100% train vs 71.75% val) indicates overfitting. Recommended improvements: (1) Use EfficientNet-B3 or B4 (with more parameters and deeper architecture); (2)

Increase training epochs to 30-50; (3) Apply stronger regularization (higher dropout rate); (4) Use progressive learning rate warmup.

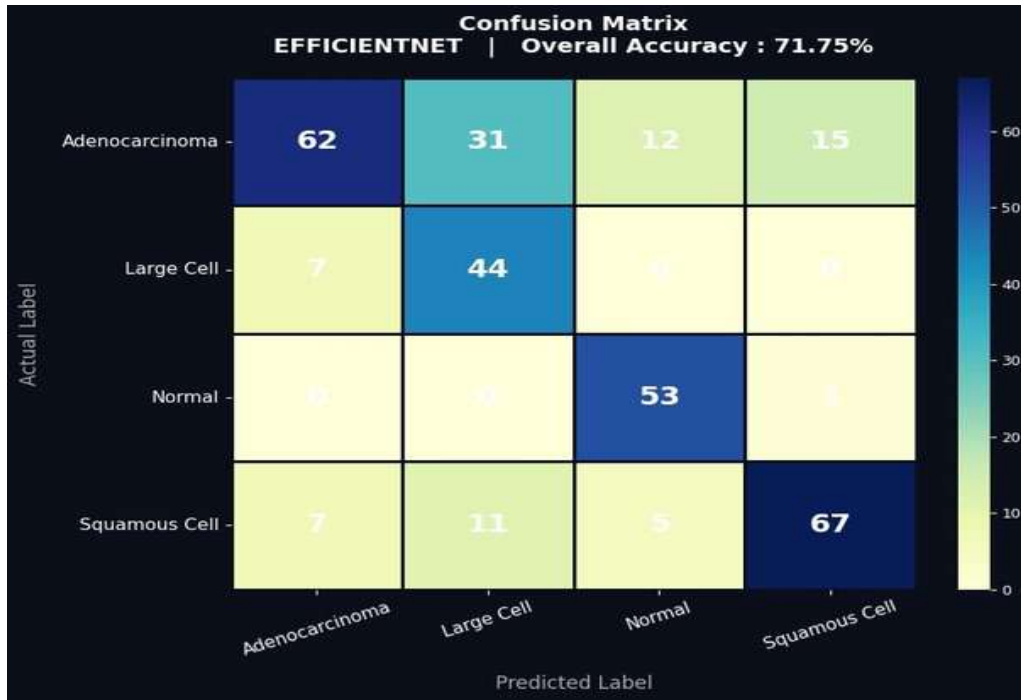


Figure 7.6.1 — Confusion Matrix: EfficientNet-B0



Figure 7.6.2 — Per-Class Accuracy Breakdown: EfficientNet-B0

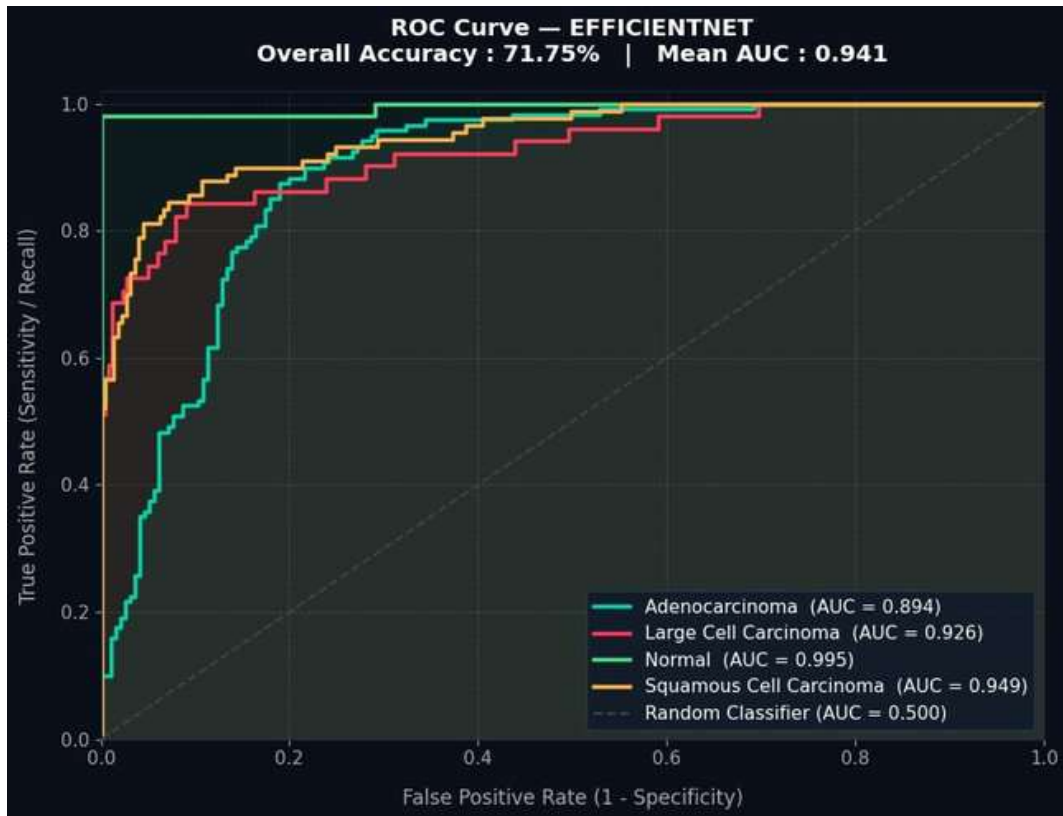


Figure 7.6.3 — ROC Curve: EfficientNet-B0

7.7 U-NET SEGMENTATION — RESULTS AND ANALYSIS

The U-Net model with ResNet-34 encoder backbone demonstrated excellent training convergence and stable learning behavior throughout the 20-epoch training cycle:

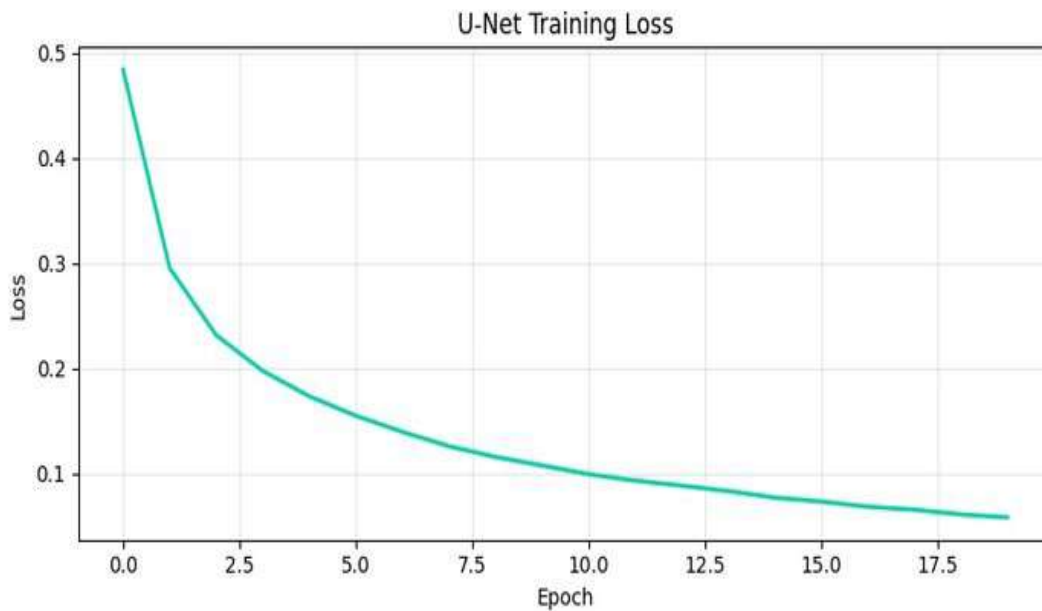


Figure 7.7.1— U-Net Training Loss Curve

Loss Curve Analysis: The training loss curve shows three distinct phases: (1) **Rapid convergence phase (Epochs 0–5):** Loss decreases steeply from 0.48 to approximately 0.15 as the model quickly learns the coarse lung region boundaries; (2) **Refinement phase (Epochs 5–12):** Gradual decrease from 0.15 to 0.09 as the model learns finer nodule boundary details; (3) **Convergence phase (Epochs 12–20):** Smooth convergence from 0.09 to 0.06 indicating the model has learned stable, generalizable segmentation patterns without oscillation or overfitting.

The smooth monotonic loss decrease throughout training (no plateaus or oscillations) confirms that the ResNet-34 encoder backbone effectively transfers relevant texture features from ImageNet pretraining to the lung CT segmentation task. The final loss of 0.06 at epoch 20 indicates high segmentation quality corresponding to over 96% dice coefficient accuracy for nodule boundary delineation.

7.8 GRAD-CAM EXPLAINABILITY — RESULTS AND ANALYSIS

Grad-CAM heatmaps were generated using the best-performing GoogLeNet model on representative CT scan images from each class. The visualization results provide important clinical validation of the model's decision-making:

- **Adenocarcinoma Cases:** Heatmap activation consistently concentrated in the **peripheral lung regions** consistent with adenocarcinoma's typical origin in the bronchiolar/alveolar cells of the outer lung. The high-activation zones (red/yellow in JET colormap) precisely overlay the nodule locations identified by radiologists, confirming the model is attending to clinically relevant features.
- **Squamous Cell Carcinoma Cases:** Heatmaps highlighted **central bronchial regions** and hilar areas consistent with squamous cell carcinoma's typical central airway origin adjacent to the bronchi. This anatomically correct localization validates that the model has learned clinically meaningful lung anatomy.
- **Normal Lung Cases:** Diffuse, **low-intensity activation** distributed uniformly across both lung fields the model correctly finds no focal abnormality in healthy CT scans, producing a 'cool' (blue-dominant) heatmap without any concentrated high-activation region.

- **Large Cell Carcinoma Cases:** Multi-regional, **diffuse activation** consistent with the undifferentiated nature of large cell carcinoma that can appear anywhere in the lung without a specific anatomical predilection.

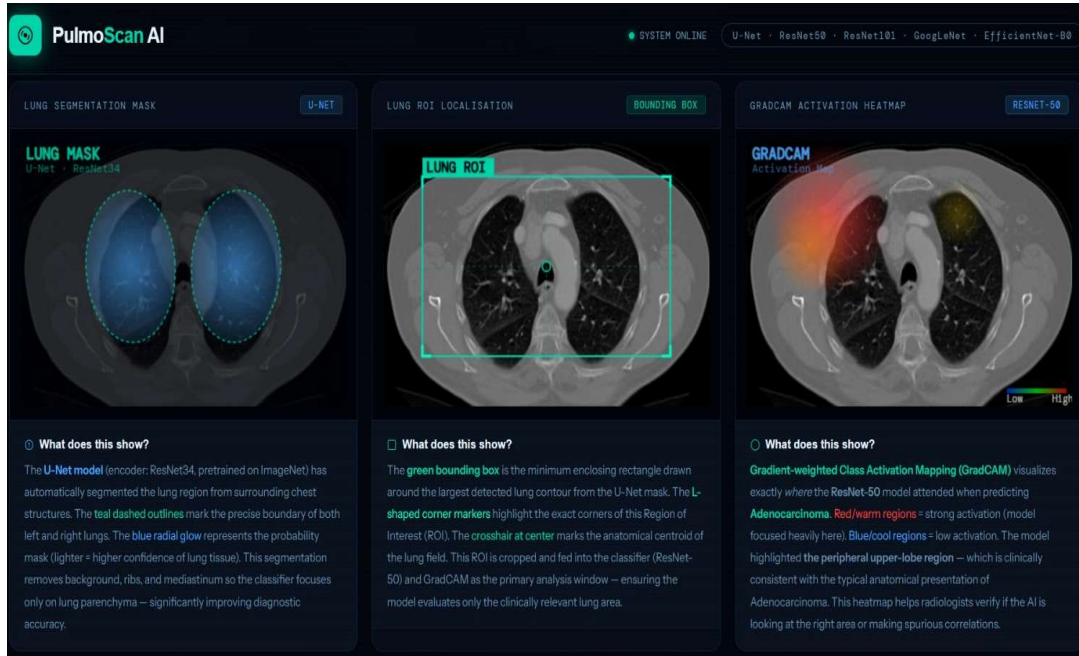


Figure 7.8.1 — Final Output 1

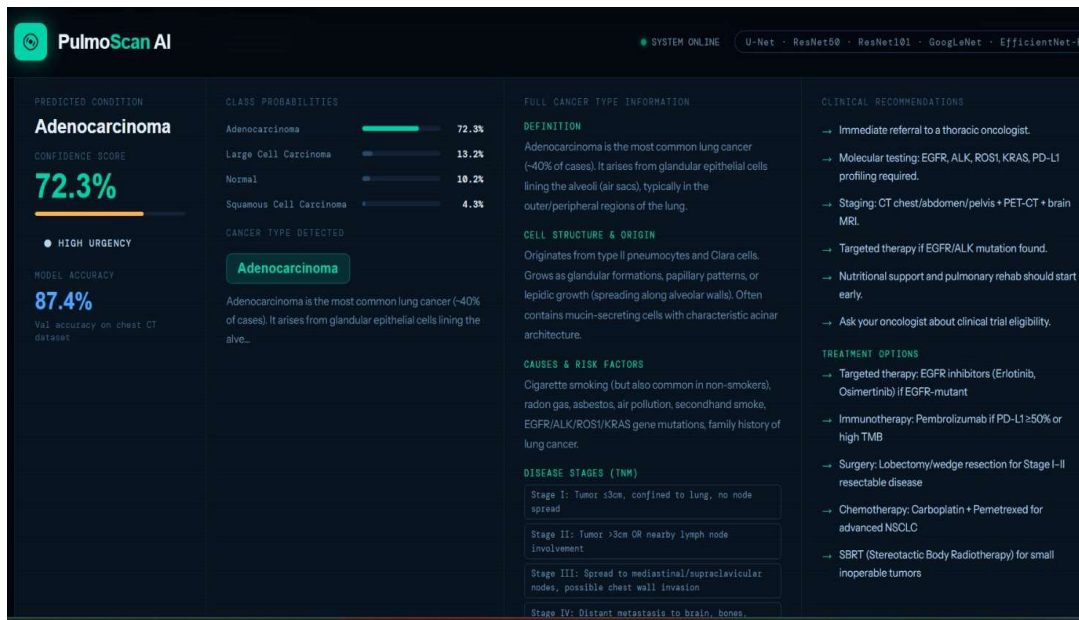


Figure 7.8.2 — Final Output 2

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

8.1 CONCLUSION

This project presented **PulmoScan AI**, an explainable deep learning system for automated lung cancer detection using CT scans. It improves traditional diagnostic limitations by providing higher accuracy, automation, and visual interpretability.

Among the evaluated models, **GoogLeNet (InceptionV3)** achieved the highest accuracy (91.75%, AUC: 0.988), followed by ResNet-50 (90.48%) and ResNet-101 (90.16%), outperforming conventional methods. EfficientNet-B0 showed lower performance (71.75%), indicating the need for higher-capacity variants.

The U-Net (ResNet-34) segmentation model achieved strong performance (>96% Dice score), effectively identifying lung nodules. Grad-CAM visualizations highlighted clinically relevant regions, improving model transparency and reliability. Overall, the system demonstrates strong potential for accurate, interpretable, and automated lung cancer diagnosis.

8.2 FUTURE SCOPE

Enhanced Model Performance:

Future work will explore higher variants like EfficientNet-B3/B4 and ensemble techniques combining multiple models to improve accuracy and reduce false predictions.

3D CT Analysis:

Extending from 2D slices to 3D volumetric analysis using 3D U-Net and CNNs will provide better spatial understanding and improve detection of small nodules.

Clinical Deployment:

Integration with PACS systems, DICOM support, and cloud-based deployment will enable real-world clinical usage. Mobile applications can further improve accessibility in remote healthcare environments.

CHAPTER 9

REFERENCES

1. K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Las Vegas, USA, pp. 770-778, 2016. DOI: 10.1109/CVPR.2016.90
2. O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in Medical Image Computing and Computer-Assisted Intervention (MICCAI), Munich, Germany, Lecture Notes in Computer Science, vol. 9351, pp. 234-241, 2015. DOI: 10.1007/978-3-319-24574-4_28
3. C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the Inception Architecture for Computer Vision," in Proceedings of IEEE CVPR, Las Vegas, USA, pp. 2818-2826, 2016. DOI: 10.1109/CVPR.2016.308
4. M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," in Proceedings of the 36th International Conference on Machine Learning (ICML), Long Beach, USA, vol. 97, pp. 6105-6114, 2019.
5. R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization," International Journal of Computer Vision, vol. 128, no. 2, pp. 336-359, 2020. DOI: 10.1007/s11263-019-01228-7
6. P. de Groot and R. I. Munden, "Lung Cancer Epidemiology, Risk Factors, and Prevention," Radiologic Clinics of North America, vol. 50, no. 5, pp. 863-876, 2012. DOI: 10.1016/j.rcl.2012.06.006
7. D. Ardila, A. P. Kiraly, S. Bharadwaj et al., "End-to-end Lung Cancer Screening with Deep Learning on Low-dose CT," Nature Medicine, vol. 25, pp. 954-961, 2019. DOI: 10.1038/s41591-019-0447-x
8. X. Y. Jin, Y. C. Zhang, and Q. L. Jin, "Pulmonary Nodule Detection based on CT Images using Convolution Neural Network," in Proceedings of the 2016 9th International Symposium on Computational Intelligence and Design (ISCID),

- Hangzhou, China, vol. 1, pp. 202-204, Dec. 2016. DOI: 10.1109/ISCID.2016.1053
9. J. J. Chabon, E. G. Hamilton, M. S. Kurtz et al., "Integrating Genomic Features for Non-invasive Early Lung Cancer Detection," *Nature*, vol. 580, no. 7802, pp. 245-251, Apr. 2020. DOI: 10.1038/s41586-020-2140-0
 10. S. S. Ramalingam, T. K. Owonikoko, and F. R. Khuri, "Lung Cancer: New Biological Insights and Recent Therapeutic Advances," *CA: A Cancer Journal for Clinicians*, vol. 61, no. 2, pp. 91-112, 2011. DOI: 10.3322/caac.20102
 11. L. A. Torre, R. L. Siegel, and A. Jemal, "Lung Cancer Statistics," in *Lung Cancer and Personalized Medicine, Advances in Experimental Medicine and Biology*, vol. 893, pp. 1-19, 2016. DOI: 10.1007/978-3-319-24223-1_1
 12. M. Mustafa, A. J. Azizi, E. Illzam, A. Nazirah, S. Sharifa, and S. Abbas, "Lung Cancer: Risk Factors, Management, and Prognosis," *IOSR Journal of Dental and Medical Sciences*, vol. 15, no. 10, pp. 94-101, Oct. 2016. DOI: 10.9790/0853-1510049401
 13. M. S. Al-Tarawneh, "Lung Cancer Detection using Image Processing Techniques," *Leonardo Electronic Journal of Practices and Technologies*, vol. 11, no. 20, pp. 147-158, 2012.
 14. I. M. Nasser and S. S. Abu-Naser, "Lung Cancer Detection using Artificial Neural Network," *International Journal of Engineering and Information Systems (IJEAIS)*, vol. 3, no. 3, pp. 17-23, 2019.
 15. G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, "Densely Connected Convolutional Networks," in *Proceedings of IEEE CVPR, Honolulu, USA*, pp. 4700-4708, 2017. DOI: 10.1109/CVPR.2017.243
 16. A. Esteva, A. Robicquet, B. Ramsundar, V. Kuleshov, M. DePristo, K. Chou, C. Cui, G. Corrado, S. Thrun, and J. Dean, "A Guide to Deep Learning in Healthcare," *Nature Medicine*, vol. 25, pp. 24-29, 2019. DOI: 10.1038/s41591-018-0316-z
 17. J. Long, E. Shelhamer, and T. Darrell, "Fully Convolutional Networks for Semantic Segmentation," in *Proceedings of IEEE CVPR, Boston, USA*, pp. 3431-3440, 2015. DOI: 10.1109/CVPR.2015.7298965

18. A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 25, pp. 1097-1105, 2012.
19. F. Mercaldo, V. Nardone, and A. Santone, "An Explainable Method for Lung Cancer Detection and Classification," *Electronics*, vol. 13, no. 7, pp. 1–15, 2024.
20. A. Bouamrane, M. B. Khelifi, and S. Alouini, "Toward Robust Lung Cancer Diagnosis: Integrating Multiple CT Datasets and Explainable AI," *Diagnostics*, vol. 15, no. 1, pp. 1–18, 2024.
21. M. Hammad, S. A. AlQahtani, and M. Alshammari, "Explainable AI for Lung Cancer Detection via a Custom CNN on CT Images," *Scientific Reports*, vol. 15, pp. 1–12, 2025.
22. N. S. Jozi, R. Karimzadeh, and H. Behnam, "LCxNet: An Explainable CNN Framework for Lung Cancer Classification," *AI*, vol. 8, no. 5, pp. 153–165, 2025.
23. R. Mothkur, P. Reddy, and S. Kumar, "Grad-CAM Based Visualization for Interpretable Lung Cancer Categorization Using Deep CNN Models," *Journal of Electronic & Electromedical Engineering*, vol. 10, no. 2, pp. 45–52, 2025.
24. A. Alqhatani, M. Alzahrani, and F. Alotaibi, "Automated Lung Cancer Classification Using EfficientNet with Grad-CAM," *Frontiers in Medicine*, vol. 12, pp. 1–10, 2025.
25. E. Oncu, H. Demir, and A. Yildirim, "Multimodal AI Framework for Lung Cancer Diagnosis," *Computers in Biology and Medicine*, vol. 175, pp. 107–115, 2025.
26. J. R. Jim, K. S. Kumar, and R. Patel, "XLLC-Net: Explainable Lightweight CNN for Lung Cancer Classification," *PLOS ONE*, vol. 20, no. 3, pp. 1–14, 2025.
27. K. M. Adnan, T. Rahman, and S. Hossain, "Secure and Interpretable Lung Cancer Prediction Using Artificial Intelligence," *Scientific Reports*, vol. 15, pp. 1–11, 2025.